IMPLEMENTING POST-QUANTUM SECURE EXOTIC SIGNATURE SCHEMES IN BLOCKCHAINS

A dissertation submitted to The University of Manchester
for the degree of Master of Science
in the Faculty of Science and Engineering

2026

Royce Steven

Department of Computer Science

Contents

Abstract **6**

Declaration **8**

Copyright **9**

Statement of qualifications and research **10**

Acknowledgements **11**

1 Introduction **12**

1.1 Context and motivation 12

1.2 Subject area and related work 13

1.3 Project category, aim, and objectives 15

1.4 Dissertation structure 16

2 Methodology **17**

2.1 The LAS construction 17

2.2 Implementation strategy: reuse, do not reinvent 18

2.3 Serialization and the on-chain interface 20

2.4 Testing methodology 20

2.5 An independent second implementation: the Rust port 21

2.6 Benchmarking methodology and fair comparison 23

2.7 Application methodology: the UTXO swap study 26

3 Results and discussion **30**

3.1 Correctness and robustness 30

3.2 Computation cost 30

3.3 Cross-implementation check: C implementation versus Rust port . 34

3.4 Communication cost and component breakdown 35
3.4.1 The price of post-quantum: classical adaptor baseline . . . 36
3.5 Application demonstration 37
3.6 The swap on a UTXO chain: three configurations compared . . . 38
3.7 Threats to validity 40

4 Evaluation **48**
4.1 Evaluation strategy and its justification 48
4.2 Achievement of the objectives 49
4.3 Implementation challenges 51
4.4 Limitations 51

5 Conclusion, critical reflection, and future work **54**
5.1 Conclusions 54
5.2 Critical reflection 55
5.2.1 What was achieved 55
5.2.2 What fell short 56
5.2.3 What would be done differently 57
5.3 Future work 57

A Code listings and reproduction **60**
A.1 Building and reproducing the benchmarks 60
A.2 The timing-benchmark procedure 62
A.3 Methodology details 63
A.4 The modelled UTXO ledger 65
A.5 Wire encoding and the validating decoder 65
A.6 The atomic-swap demonstration and its proof of knowledge 66
A.7 The optimistic (Naysayer) on-chain verifier 67
A.8 Full benchmark data tables 68
A.9 Auditing the reuse claim 68
A.10 Selected code listing: Adapt and Extract 69

Bibliography **72**

Word Count: 9658

List of Tables

2.1 Source files: reused, modified, and added 19
2.2 The four test types 21
2.3 The three measurement boundaries 23
2.4 Parameter sets used in the evaluation 26
2.5 Security and scheme parameter notation 27
2.6 The three measured configurations 28

3.1 The adaptor-signature correctness contract 31
3.2 Adaptor overhead at the target setting, both timing tiers 32
3.3 Rejection-sampling statistics: model vs measurement 33
3.4 Per-operation timing: C implementation vs Rust port 35
3.5 Serialized size of every LAS object 36
3.6 Classical vs. post-quantum adaptor signature 44
3.7 On-chain settlement gas: classical vs. post-quantum 45
3.8 Per-phase execution time of the swap 46
3.9 Communication cost of the swap 47

4.1 Objectives against evidence 50
4.2 The six implementation challenges 52

A.1 Per-operation timing across the parameter sets 70
A.2 LAS objects by component (packed bytes) 71

List of Figures

2.1 The LAS adaptor data flow 18
2.2 Repository structure: two implementations over reused primitives 22

3.1 Adaptor overhead per operation: basic signature vs LAS 41
3.2 Cumulative probability of acceptance at the target setting 42
3.3 Criterion.rs sampling distribution for PreSign 43
3.4 On-chain settlement gas for the three claim paths 46

A.1 Adaptor overhead across the parameter sets (sensitivity check) . . 69

Abstract

Implementing Post-Quantum Secure
Exotic Signature Schemes in Blockchains
Royce Steven
A dissertation submitted to The University of Manchester
for the degree of Master of Science, 2026

Blockchains authenticate transactions with elliptic-curve signatures, which a large quantum computer would break. Basic post-quantum signatures are now standardised, but the *exotic* signatures that give blockchains their advanced features — in particular *adaptor signatures*, the cryptographic core of atomic swaps and payment channels — remain largely paper-only in the post-quantum setting. This dissertation implements and evaluates one such scheme, LAS, a lattice-based adaptor signature, by reusing the CRYSTALS-Dilithium reference primitives, and demonstrates it end-to-end in a post-quantum atomic swap.

The artefact is built *additively*. The entire lattice core — polynomial arithmetic, the number-theoretic transform, and sampling based on SHAKE (an extendable-output hash function) — is reused unchanged, verified by a file-level diff against a pristine baseline; not a single upstream function was modified. The adaptor layer (PreSign, PreVerify, Adapt, Extract), a bit-packed wire encoding with a validating decoder, and the application are added as new code. The implementation passes functional tests over 1000 iterations on three parameter sets, exact serialization round-trips, a tamper test in which all 4640 single-byte signature flips are rejected, and pinned known-answer tests. An independent second implementation in Rust, built the same way, reproduces the wire format and the pinned digest byte-for-byte and confirms the benchmark conclusions under a different compiler and statistical

harness.

The primary, fair comparison is LAS against its own simplified Dilithium-style base at identical parameters and primitives, isolating the adaptor layer's cost; the optimised CRYSTALS-Dilithium reference is reported only as context, and a classical Elliptic Curve Digital Signature Algorithm (ECDSA) adaptor as a functionality-matched "price of post-quantum" baseline. Every timing is a mean over repeated runs, reported with its standard deviation at an explicitly declared measurement boundary. At the core boundary the adaptor layer is almost free in computation: PreSign, PreVerify and Adapt each add at most +10.5% over the basic operation they mirror, at every parameter set, and Extract is the cheapest operation of all. The real cost is *communication* — the signature is 99.3–99.7% lattice response vector z, and an adaptor swap additionally transmits a public-key-sized statement. That cost reappears as computation once the byte boundary is included: wire encoding and decoding raise the adaptor premium to +20.8–79.5%.

Taking the protocol to a UTXO ledger, the venue the original construction assumes, sharpens the conclusion: at the application level the signature is not the dominant cost at all. Measured across three configurations, the zero-knowledge proof of knowledge consumes over 98% of a swap, and substituting a lattice prover for a pairing-based one makes the swap *faster* while making it larger. The dissertation concludes that a working, tested, end-to-end post-quantum exotic signature is achievable by reuse of standardised primitives; that entropy coding of the response and migration to the original construction's larger modulus remain the tractable size reductions, while the Dilithium optimisations this scheme had to disable pose an open compatibility problem; and that the proof system, not the signature, is where a post-quantum swap should next be optimised.

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

Statement of qualifications and research

[TODO: Only you can write this — state your prior degree(s)/qualifications and any relevant research experience, e.g. "The author holds a BSc in ... from .... No part of the work reported here has been undertaken as part of any previous qualification."]

Acknowledgements

[TODO: Optional but conventional — e.g. thanks to your supervisor Dr Zhipeng Wang, and to anyone else you wish to credit.]

Chapter 1

Introduction

This chapter sets out why post-quantum exotic signatures matter for blockchains, what this project builds, and how the dissertation is organised. It first establishes the context, then reviews the necessary literature, then states the aim and objectives.

1.1 Context and motivation

Public blockchains authorise every transaction with a digital signature, almost always the Elliptic Curve Digital Signature Algorithm (ECDSA), Schnorr, or Boneh–Lynn–Shacham (BLS) signatures over an elliptic curve. The security of all three rests on the hardness of the elliptic-curve discrete logarithm problem. Shor's algorithm solves that problem in polynomial time on a sufficiently large quantum computer [17], so a future quantum adversary could forge signatures and spend other users' funds. "Post-quantum" (PQ) cryptography replaces these assumptions with problems believed hard even for quantum computers, such as those over lattices.

The U.S. National Institute of Standards and Technology (NIST) has standardised *basic* PQ signatures, notably ML-DSA (the Module-Lattice-Based Digital Signature Algorithm), derived from CRYSTALS-Dilithium [13, 5]. A basic signature authenticates a message and nothing more. Blockchains, however, depend on *exotic* signatures that add functionality on top of a basic scheme: multi-signatures, aggregate signatures, threshold signatures, and *adaptor signatures*. These exotic schemes are what give blockchains scalable consensus, compact multi-party authorisation, and advanced payment protocols. A recent survey documents that, while

basic PQ signatures are maturing, their exotic counterparts largely lack efficient, practical implementations in a blockchain setting [3]. Closing that gap — turning paper designs into tested, measured, blockchain-facing artefacts — is the problem this project addresses.

This dissertation focuses on the *adaptor signature*, which ties the act of publishing a valid signature to the simultaneous revelation of a secret witness [14, 1]. That single primitive underpins "scriptless" atomic swaps and payment-channel networks: it lets two parties exchange assets, even across different chains, so that either both transfers happen or neither does, without trusted intermediaries or heavy on-chain scripts. In the classical setting these constructions are mature and deployed; in the post-quantum setting they are mostly paper-only, which makes the adaptor signature a representative, high-value exotic scheme to implement and evaluate.

The adaptor signature is best understood through the history of the atomic swap. A cross-chain exchange first relied on custodial intermediaries; hash-time-locked contracts (HTLCs) removed the custodian by locking both transfers under the preimage of one hash, so that claiming one leg reveals the preimage unlocking the other. HTLCs, however, need explicit on-chain script support, place the common hash on both chains — explicitly linking the two legs and, in multi-hop payments, every hop of a path — and enlarge the transactions. The adaptor signature answers these limitations [14, 1] by moving the lock *into the signature itself*, so settlement transactions look like ordinary single-signer payments — no script beyond signature verification, no shared value linking the legs on chain, no extra on-chain bytes — while retaining atomicity and revealing the witness only to the counterparty holding the matching pre-signature. The link removed is the explicit protocol one: timing, amounts and transaction-graph analysis can still correlate the two legs.

1.2 Subject area and related work

This section reviews only the work needed to understand the problem and justify the approach; it is deliberately not an exhaustive survey, in line with the report guidance to favour depth over breadth.

Exotic signatures for post-quantum blockchains. The motivating survey [3] catalogues exotic signature families — multi-, aggregate, threshold, ring, blind, and adaptor signatures — and identifies the shortage of efficient PQ-secure blockchain implementations as an open challenge. This project contributes one such implementation and evaluation.

Lattice signatures and Dilithium. CRYSTALS-Dilithium is a lattice signature in the "Fiat–Shamir with aborts" family [5, 11], its security reducing to the Module Learning With Errors (Module-LWE) and Module Short Integer Solution (Module-SIS) problems. It samples a masked commitment, derives a challenge by hashing, computes a response, and *rejects and retries* when the response would leak the secret — the rejection-sampling step characterising the family. Its reference implementation provides the polynomial arithmetic, number-theoretic transform (NTT) and SHAKE-based sampling (SHAKE being an extendable-output hash function) that this project reuses.

Adaptor signatures. Adaptor signatures were introduced informally as "scriptless scripts" [14] and later formalised, with applications to channels and swaps [1]. The abstraction adds four algorithms to a base signature — PreSign, PreVerify, Adapt, Extract — relative to a hard relation (Y, y): a pre-signature can be *adapted* into a full signature by anyone knowing the witness y, and publishing that signature lets anyone holding the pre-signature *extract* y. Anonymous multi-hop locks generalise this to payment paths [12].

Post-quantum adaptor signatures. Esgin, Ersoy and Erkin proposed LAS, a lattice-based adaptor signature on a Dilithium-style base, with PQ payment-channel constructions [6]. LAS is the design implemented here. Their work is primarily theoretical — definitions, a construction and security proofs — so a practical, benchmarked, blockchain-facing implementation is this dissertation's contribution. Recent work such as poqeth studies how PQ primitives are integrated and costed on Ethereum [9], providing a template for the on-chain side.

Classical baseline. The natural answer to "what does post-quantum cost" is a comparison with a *classical adaptor* signature — not merely plain ECDSA. Mature ECDSA-adaptor implementations exist [2]; this project benchmarks against one,

and against Dilithium itself, taking care (see chapter 2) to state security parameters explicitly so that the comparisons are fair.

1.3 Project category, aim, and objectives

Category. This is a *systems implementation and evaluation* project grounded in existing research, not a proposal of a new cryptographic protocol; following the project brief, its emphasis is practical implementation and benchmarking rather than theoretical design. The contribution is a reliable, tested, benchmarked realisation of an existing exotic scheme (LAS, [6]) and its demonstration in a blockchain atomic-swap application. A genuinely new research question would require adding functionality on top of adaptor signatures — functional adaptor signatures, say — which is noted as future work but out of scope here.

Aim. To implement and evaluate a post-quantum exotic signature scheme — a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives — and to demonstrate it in a post-quantum blockchain atomic-swap application.

Objectives.

1. Review post-quantum and exotic-signature literature sufficiently to justify selecting the adaptor signature LAS and the Dilithium primitives.

2. Implement LAS additively on the Dilithium reference, reusing the lattice core unchanged and adding the adaptor operations and a byte-level wire encoding.

3. Validate correctness and robustness through functional, serialization, tamper, and known-answer tests, including cross-validation by a second, independent implementation checked against the same known-answer digest.

4. Benchmark LAS fairly — at explicitly stated parameters, with repeated runs and reported variance — against a simplified Dilithium baseline at identical parameters, the NIST-optimised Dilithium, and a classical ECDSA-adaptor baseline, separating computation from communication cost and reporting key-generation time, public-key size, signature size, signing time, and verification time.

5. Demonstrate an end-to-end post-quantum atomic swap using the implementation — following the protocol of [6] verbatim, including its zero-knowledge proof of knowledge π — and discuss its feasibility.

1.4 Dissertation structure

Chapter 2 describes the methods: the LAS construction, the reuse-based implementation strategy, the testing approach, and the benchmarking methodology. Chapter 3 reports the test and benchmark results and discusses their validity. Chapter 4 justifies the evaluation strategy and judges the work against the objectives. Chapter 5 concludes, reflects critically on the approach, and proposes future work.

Chapter 2

Methodology

This chapter explains how the work was done: the construction, the reuse-based implementation strategy, the testing regime, the independent Rust port that cross-validates it, and — decisive for a fair evaluation — the benchmarking methodology and parameter choices. The *measured* UTXO study, which carries real experimental design, is set out in section 2.7.

2.1 The LAS construction

LAS is a lattice adaptor signature whose base is a simplified, Dilithium-style Fiat–Shamir-with-aborts signature [6, 5]. All objects live in the ring $R_q = \mathbb{Z}_q[X]/(X^d + 1)$, the public matrix has the form $A = [\, I_n \mid A' \,] \in R_q^{n \times (n+\ell)}$, and a key pair is a short secret r with its image $t = A \cdot r$. The hard relation (Y, y) reuses exactly that structure, so a statement is "just another public key": recovering its witness means finding a short preimage of Y under A, a Module-SIS problem, while Module-LWE keeps the witness hidden [6]. Crucially *key generation is shared* — LAS reuses the base KeyGen unchanged and the statement/witness pair comes from that same generator — so LAS is exactly the base signature plus four adaptor operations, which is why every comparison here pairs a LAS operation with the base operation it extends.

The base signature samples a masked commitment w, forms a challenge $c = H(pk, w, M)$, computes a response and rejects when $\|\mathbf{z}\|_\infty > \gamma - \kappa$. Relative to a statement Y, the adaptor extension [6] adds PreSign, which folds the statement into the hash, $c = H(pk, w + Y, M)$, and rejects at the *tighter* bound $\gamma - \kappa - 1$; PreVerify, which recomputes $w' = A\mathbf{z} - c\,t$ and checks $c = H(pk, w' + Y, M)$;

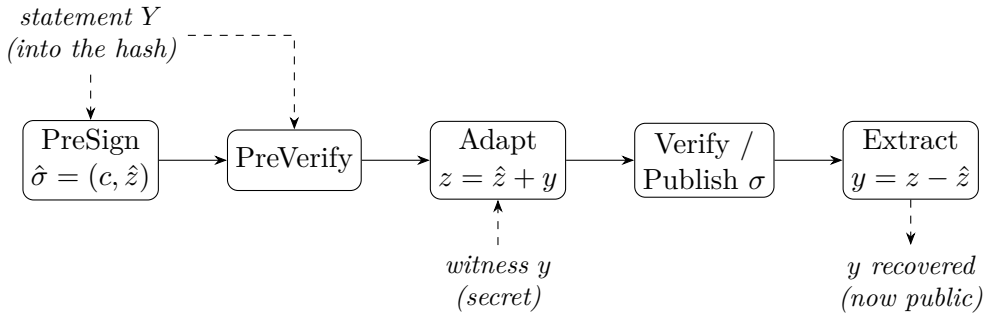

Figure 2.1: The LAS adaptor data flow. The statement Y is folded into the challenge hash during PreSign and PreVerify; the witness y is added by Adapt to turn the pre-signature into a base signature; and publishing that signature lets anyone holding the pre-signature run Extract to recover y. This last step is what makes an atomic swap atomic.

Adapt, which adds the witness, $\mathbf{z} \leftarrow \hat{\mathbf{z}} + y$, producing a signature the base verifier accepts; and Extract, which recovers $y = \mathbf{z} - \hat{\mathbf{z}}$.

$$\text{accept iff } \|\mathbf{z}\|_\infty \le \gamma - \kappa, \qquad \gamma = \kappa\, d\,(n + \ell). \tag{2.1}$$

The single subtle design point is the bound budget. The witness is ternary, so $\|y\|_\infty \le 1$, and because PreSign rejects at $\gamma - \kappa - 1$ the response after Adapt satisfies $\|\mathbf{z}\|_\infty \le \gamma - \kappa$ and clears eq. (2.1). Loosening PreSign to $\gamma - \kappa$ would let adapted signatures exceed the bound, and the base verifier would then reject them. Figure 2.1 shows the data flow and, crucially, where Y enters and where y is revealed.

2.2 Implementation strategy: reuse, do not reinvent

The guiding principle is that an exotic scheme equals a basic scheme plus extra functions, so the lattice arithmetic should be *reused*, not rewritten. The implementation is built additively on the upstream CRYSTALS-Dilithium reference [5, 4] at a pinned commit, and the same strategy is applied a second time in Rust (section 2.5). Neither is itself "the reference": both are *modified, simplified* ML-DSA-style bases carrying the LAS layer of [6], and are named the *C implementation* and the *Rust implementation* throughout.

Table 2.1: Source files of the implementation, classified by how the reference is used. The lattice core is reused unchanged; the only modified file is the `Makefile`; the scheme, serialization, application, and evaluation are new.

Category	Files	Role
Reused unchanged	`poly/polyvec/ntt/reduce/rounding`, `packing`, `sign`, `fips202`, `params`	the entire lattice core
Modified	`Makefile`	compile new targets
Added	`las.{c,h}`, `serialize.{c,h}`, `relation_zk.{c,h}` (+ LaZer bridge), `amhl`, `chain`, tests, benchmarks	this work

Table 2.1 summarises the classification: no upstream source function is modified, so the security-critical lattice arithmetic is exactly the published reference code, every adaptor-specific line is new and isolated, and the boundary between them is unambiguous. The same posture extends to the vendored third-party libraries — `secp256k1-zkp` [2] for the classical baseline and the LaZer library [10] for the proof of knowledge π, both unmodified, the latter behind a single-file bridge.

Data types and the matrix product. LAS is a self-contained module over the reference's degree-d polynomial type. Because $A = [\, I_n \mid A' \,]$, the product $A\,v = v_{\text{top}} + A' v_{\text{bot}}$ adds the identity block directly and multiplies only A' in the NTT domain — saving work and matching how the reference itself computes As_1.

Hash, challenge, and samplers. The random oracle H is built from SHAKE256, absorbing a canonical encoding of the public key, then the commitment (w for Sign, $w + Y$ for PreSign), then the message. The challenge sampler is the reference's `SampleInBall` at weight κ (table 2.4), guaranteeing $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Two samplers are added: a ternary sampler for secrets and witnesses, and a rejection sampler uniform on $[-\gamma, \gamma]$ for the mask, whose draw width is the smallest $2^k - 1 \geq 2\gamma$ and is therefore 18 or 19 bits depending on the parameter set. Norm checks reuse the reference's `poly_chknorm`.

Simplifications, and the modulus deviation. Following [6], the base signature omits the full Dilithium scheme's hint vector, Power2Round and high/low-bit decomposition, hashing the full commitment w rather than its high bits. This keeps the exact identity $A\mathbf{z} - c\,t = w$ available, which the adaptor mechanism

requires, at the cost of larger keys and signatures (quantified in chapter 3). Separately, [6] specifies $q \approx 2^{24}$, but reusing the reference NTT fixes the root-of-unity table to $q = 8\,380\,417 \approx 2^{23}$; since $q > 2\gamma$ every intermediate value is represented faithfully and correctness is unaffected, and only the concrete Module-SIS/LWE margin differs. Migrating would require a new NTT table and is future work (section 5.3). No module depends on a mode-specific constant, so identical code runs at every parameter set.

2.3 Serialization and the on-chain interface

A deployable scheme exchanges *bytes*, not in-memory structs, so a bit-packed wire encoding, a *validating* decoder and a verify-from-bytes entry point (`base_verify_packed`) were implemented. Two decisions matter for what follows. The decoder is *defensive*, rejecting out-of-range public-key coefficients, the invalid ternary code and out-of-band response fields, because a verifier cannot trust its input; this is the interface a Solidity, precompile or circuit verifier would expose, and the object the tamper test attacks (section 3.1). And, following FIPS 204 [13], a signature carries not the challenge polynomial c but the 32-byte hash $\tilde{c}$ from which every consumer re-derives it — a computation-for-storage trade that keeps signatures smaller and matches the standard. Appendix A.5 gives the field widths and the full decoder contract.

2.4 Testing methodology

Correctness and robustness were established with the four test types of table 2.2, chosen to cover progressively weaker assumptions about the environment: functional tests assume an honest caller, serialization and tamper tests a hostile one, and the known-answer test a different machine entirely. All pass, and the implementation compiles with no warnings under a strict set of diagnostics.

Table 2.2: The four test types, what each asserts, and what it would catch. The functional suite's statement-binding clause — that a pre-signature *fails* basic verify — is the tripwire that distinguishes an adaptor signature from a signature; the known-answer digest is what the second implementation is checked against (section 2.5).

Test type	Scale and assertions	Catches
Functional	1000 iterations per Dilithium mode (2, 3, 5), fresh parameters, keys and message each time; asserts the full adaptor cycle — pre-signature pre-verifies, *fails* basic verify, adapts to a valid signature, and extraction returns the witness exactly — plus a sign/verify round trip and one-bit-forgery rejection	any break in the adaptor contract; statement binding
Serialization	256 random instances; exact round-trip for keys and every signature variant	encoder/decoder asymmetry
Tamper	every one of the 4640 bytes of a valid packed signature flipped in turn; all must break verification; malformed encodings must be rejected	a decoder that trusts its input
Known-answer	all inputs fixed, deterministic API, packed bytes of every object folded into one SHAKE256 digest compared against a pinned 32-byte value	any unintended change to sampling, algebra or encoding, on any machine

2.5 An independent second implementation: the Rust port

The complete scheme — base signature, four adaptor operations, wire encoding and deterministic known-answer interface — was implemented a second time in Rust, on the same reuse principle: layered on a fixed version of the `fips204` crate [16], reusing its polynomial arithmetic, number-theoretic transform and SHAKE-based sampling, and adding the LAS layer as new modules. The wire format is byte-identical to the C encoder's by construction, and fig. 2.2 shows the resulting repository at a glance.

The port serves three purposes. *Cross-validation*: short of a formal proof, the strongest practical argument that a cryptographic implementation is correct

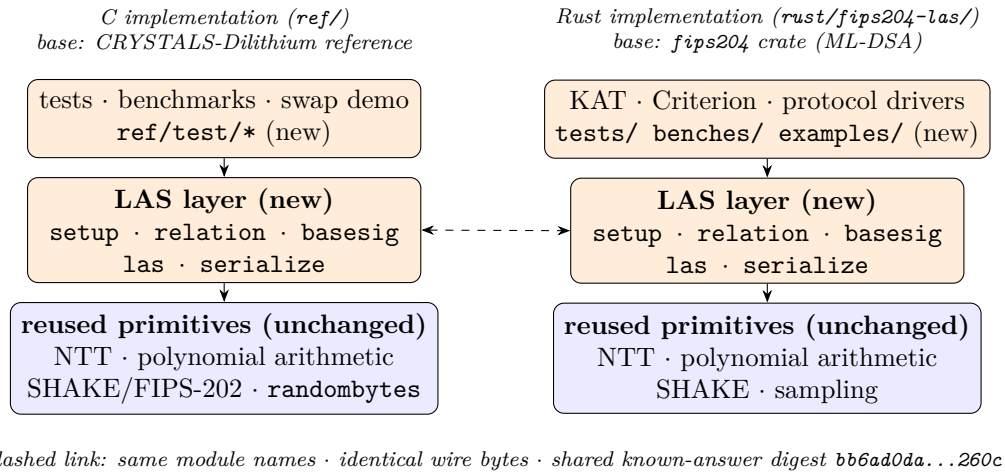

Figure 2.2: Repository structure. Both implementations have the same shape: a new, self-contained LAS layer (orange) — the same five modules under the same names — built on an *unmodified* base of reused lattice primitives (blue), with the tests and benchmarks on top. The two columns are tied together by construction: identical wire bytes and the shared pinned known-answer digest (`bb6ad0da...260c`). This is the high-level map for reading the code; the per-file classification is table 2.1.

is a second one written against the specification rather than the first's quirks, agreeing on pinned vectors. *An independent benchmark methodology*: Criterion.rs [8] measures the same operations without sharing a line of timing code with the C harness, so agreement defends the C numbers against harness artefacts. *Deployment realism*: a memory-safe language is the plausible vehicle for production use.

To make the timings comparable the two protocol drivers mirror each other exactly — same repetition scheme, fixed seed, fixed message, same success-path gating, same rejection statistics. One caveat qualifies the port's independence: where [6] leaves an engineering choice open, the Rust implementation deliberately adopts the C implementation's, concretely that the signing loop's rejection check *aborts early* at the first out-of-bound coefficient rather than scanning the whole response vector. Sharing that is what makes the two languages time the same algorithm. The tie is established by direct inspection of the two norm checks, not by the automated evidence: an early-abort and a full-scan check return identical verdicts on identical inputs, so neither the shared digest nor the attempt-count gate could detect a divergence — they constrain outputs and rejection rates, not the work profile of a rejected attempt.

Table 2.3: The measurement boundaries. Comparisons are only ever made within one boundary. The classical column is not a uniform tier but a *hybrid*, because `secp256k1-zkp` exposes the pre-signature only in packed form: the four adaptor operations behave like the packed tier and the three base operations like the core tier.

Boundary	What crosses it	What it answers
Core tier	in-memory structures in, structures out	what the adaptor *algebra* costs, isolated from encoding
Packed tier	packed wire bytes in and out, including the validating decode of every input and encode of every output (section 2.3)	what a wire or on-chain consumer pays per call with nothing cached; one operation's byte boundary, *not* a whole-protocol cost
Hybrid native API (classical only)	the boundary `secp256k1-zkp` itself exposes: PreSign, PreVerify, Adapt and Extract pack or parse the 162-byte pre-signature *inside* the timed call; KeyGen, Sign and Verify exchange parsed structs with the wire codecs outside it	the classical cost as the unmodified library actually charges it

2.6 Benchmarking methodology and fair comparison

The evaluation separates *computation* cost (wall-clock time per operation) from *communication* cost (bytes transmitted or stored) and never conflates them.

Measurement boundaries. Reported timings depend critically on where the measurement boundary is drawn — undocumented boundaries are a recognised benchmarking pitfall for lattice signatures [15]. Every timing therefore belongs to one of the boundaries defined in table 2.3, comparisons are only ever made within one, and wherever a base-versus-LAS comparison is presented *both tiers are shown*. The classical baseline admits neither cleanly, its library never exposing an unpacked pre-signature and modifying it would break the reuse-unmodified posture of section 2.2; its *hybrid* boundary is reported as-is, per operation (table 3.6).

Implementation of record, repeatability, and machine. Each measurement states its origin: unless a caption says otherwise, timings are the C implementation's, with the Rust results reported separately (table 3.4). The two are never averaged — different toolchains make them samples from different populations — while communication sizes, byte-identical by construction, apply to both. Every timing is the mean over 5 repetitions of 500 iterations per sign-class operation and 1000 per verify-class operation, reported as mean ± sample standard deviation. All non-random inputs are fixed and stated, so run-to-run variability comes only from rejection sampling and machine noise, never from input selection [15]; the *randomised* signing paths are the ones timed, the deterministic API existing only for the known-answer tests. All measurements were taken on one machine at fixed library versions, with the toolchain, hardware and per-table commands recorded in appendix A.1.

Per-operation reporting, not cumulative. Every operation is timed independently rather than as one end-to-end figure, because the operations are split across parties — one signs or pre-signs, another verifies and later adapts, a third may extract — so a combined figure would average over costs never paid together, and would hide the overhead operation by operation.

The primary comparison: LAS against its own simplified base. LAS is exactly its base signature plus four operations, on the same code, parameters and primitives, so holding everything fixed except the adaptor layer measures its *overhead* and nothing else — the only same-algorithm, same-parameter, same-primitive comparison available. Each adaptor operation is paired with the base operation it mirrors: PreSign against Sign, and both PreVerify and Adapt against Verify, since Adapt runs a PreVerify before adding the witness. Extract is reported separately, having no base analogue.

Rejection sampling: the expected attempt count. Sign-class operations restart until the response passes its norm bound, so their cost is a random multiple of one attempt's cost and no timing claim is meaningful without the attempt statistics. Each coefficient is accepted with probability exactly $\big(2(\gamma - \kappa) + 1\big)/(2\gamma + 1)$ *independently of the secret* — derived in appendix A.3, and precisely why rejection sampling makes the response simulatable. An attempt

succeeds only if all $(n+\ell)\,d$ coefficients pass:

$$p_{\text{acc}} = \left(\frac{2(\gamma-\kappa)+1}{2\gamma+1}\right)^{(n+\ell)d} \approx \left(1-\frac{\kappa}{\gamma}\right)^{(n+\ell)d} \approx e^{-\kappa(n+\ell)d/\gamma} = e^{-1} \approx 36.8\%, \tag{2.2}$$

the last step using the design choice $\gamma = \kappa\, d\,(n+\ell)$, made in [6] exactly so the expected number of restarts is "about $e < 3$". The attempt count is geometric, so expected attempts per call are $1/p_{\text{acc}}$: at the target setting, **2.719 for Sign** and **2.775 for PreSign**, whose bound is tighter by exactly one. PreSign is thus *expected* to restart about 2% more often before any adaptor arithmetic is counted, so the measured counters are reported beside the timings and the two effects can be separated.

Rejection sampling: the run-validity gate. Because the attempt count is geometric, per-call timings are heavy-tailed and averages over too few calls can drift several percent on rejection luck alone [15]. The drivers therefore treat the attempt statistics as a *validity gate*: the measured mean attempts must match the closed-form expectation within five standard errors, separately for Sign and PreSign, at both tiers, in C and Rust, and a run that fails aborts instead of producing evidence. Passing it provides evidence that the observed rejection behaviour is consistent with theory and that rejection-sampling variation is sufficiently controlled for the reported mean — a consistency check on the rejection rate, not a proof of sampler correctness (appendices A.2 and A.3). A second gate precedes every timing: the correctness contract of section 2.4 is asserted on the very objects about to be timed, so no failure path is ever measured.

Parameter sensitivity and component sizes. To test whether the adaptor overhead scales with the lattice dimensions, the same comparison is repeated at the four parameter sets of table 2.4, whose symbols table 2.5 defines. Their module dimensions come from the standard Dilithium choices at NIST levels 2, 3 and 5 — used purely as established, well-studied lattice dimensions to scale across — alongside those of the original construction; the parameters being compile-time constants, identical code runs at every set. No concrete bit-security is claimed, security proofs being out of scope. Each key and signature is additionally decomposed into components, so the *cause* of a size difference can be identified rather than merely its total stated.

Table 2.4: Parameter sets used in the evaluation. The *LAS-2020/845 reference* set is the parameter set proposed in [6]; the *Simplified Dilithium-II/III/V* sets reuse the dimensions of Dilithium modes 2/3/5 (so the public-key, secret, and challenge dimensions line up with NIST levels 2/3/5) and are engineering benchmark settings, *not* formal NIST/ML-DSA security-equivalence claims. All lattice sets share the ring degree $d = 256$ and modulus $q = 8\,380\,417 \approx 2^{23}$ inherited from the reused Dilithium NTT; *Simplified Dilithium-III* is the project's target set. The classical secp256k1 ECDSA adaptor is a functionality-matched baseline at ≈128-bit classical security. Table 2.5 defines each symbol.

Setting	n	ℓ	M	κ	γ	d	q
LAS-2020/845 reference	4	4	8	60	122 880	256	8 380 417
Simplified Dilithium-II	4	4	8	39	79 872	256	8 380 417
Simplified Dilithium-III (target)	6	5	11	49	137 984	256	8 380 417
Simplified Dilithium-V	8	7	15	60	230 400	256	8 380 417
Classical (secp256k1 ECDSA adaptor)	≈128-bit classical (see table 3.6)						

The classical baseline: functionality-matched, not security-matched. The second baseline is the `secp256k1-zkp` library's `ecdsa_adaptor` module [2], implementing Fournier's one-time verifiably-encrypted-signature construction [7], used unmodified at a pinned commit. This is *not* a same-security comparison — secp256k1 offers approximately 128-bit *classical* security and has no post-quantum level — but a functionality-matched "price of post-quantum" baseline: both schemes provide adaptor functionality for scriptless swaps, on different hardness assumptions. Plain ECDSA is excluded, the fair counterpart of an adaptor signature being another adaptor signature.

2.7 Application methodology: the UTXO swap study

The narrated demonstration of section 3.5 shows the protocol *runs*. A second, separately designed study measures what it *costs* and attributes that cost to a component. Because its conclusions depend entirely on what is held fixed, its design is stated here rather than beside its results.

Table 2.5: Security and scheme parameters: the symbol, its meaning, and the value(s) it takes across the parameter sets, *listed in the row order of* table 2.4 (LAS-2020/845 reference; Simplified Dilithium-II; -III; -V). The notation follows the LAS paper [6]: the ring degree is its d, and this build runs at $d = 256$, the ring degree of the reused Dilithium NTT. The figure and table captions in chapter 3 refer back to these names and values.

Symbol	Meaning	Value(s): LAS-2020/845 reference, Simplified Dilithium-II/III/V
n	module rank: rows of A, length of public key t	4, 4, 6, 8
ℓ	extra columns of A' (secret-vector extension)	4, 4, 5, 7
$M = n + \ell$	response dimension (number of polynomials in $z, \hat{z}, y$)	8, 8, 11, 15
κ	challenge Hamming weight (number of ± 1 in c; Dilithium τ)	60, 39, 49, 60
γ	rejection / masking bound, $\gamma = \kappa\, d\, (n + \ell)$	122 880, 79 872, 137 984, 230 400
d	ring degree $(X^d + 1)$	256 (all sets)
q	modulus from the reused NTT, $q \approx 2^{23}$	8 380 417 (all sets)

The venue, and why it is modelled. The construction [6] states its applications over "a UTXO-based blockchain like Bitcoin *where the signature algorithm is replaced with a lattice-based signature scheme*". That sentence is load-bearing and is implemented literally: a UTXO ledger was built in which *the signature algorithm is a parameter*, so one ledger serves every configuration and any measured difference is the cryptography, not the ledger. It is a model rather than a node, and appendix A.4 states exactly what it does and does not supply — real transaction objects, canonical serialisation and the public leak channel on which atomicity depends, but no fee market, propagation or consensus — and why deploying the post-quantum configurations on a real Bitcoin node is blocked on chain support rather than on effort.

Three configurations, and what is controlled. The three configurations of table 2.6 each run Fig. 1 of [6] end-to-end across two ledgers. The *role-A* proof is the proof of knowledge π that Fig. 1 places on the wire immediately after the statement Y: its author is the party that ran Gen, and its claim is that a witness to Y exists and is short. Only the $2 \to 3$ step is controlled, and the design works

Table 2.6: The three configurations. Only the $2 \to 3$ step is controlled: those two share one public matrix, prove the identical relation, and receive byte-identical inputs from one pinned seed, so their difference is the proof system alone. Configuration 1 carries *no* role-A proof, and this is a finding rather than an omission: LAS needs π because of its knowledge gap — extraction returns a witness of the extended relation, which need not be ternary, so without π a malicious u_1 could claim and leave u_2 holding an unadaptable pre-signature — whereas an ECDSA adaptor has no such gap, since extraction recovers the discrete logarithm exactly. Deployed classical swaps accordingly carry no π, and adding one would have benchmarked a construction nobody runs. The DLEQ proof carried *inside* a `secp256k1-zkp` pre-signature is a different object and is not counted as π: its author is the pre-signer, and its claim is pre-signature well-formedness, not knowledge of the role-A statement witness r'. Its cost is reported in the bytes by which a pre-signature exceeds a signature.

#	Signature	Role-A proof π	Setup	Fully PQ
1	ECDSA adaptor signature (libsecp256k1-zkp)	none required	—	no
2	LAS (simplified Dilithium-III)	Groth16 over BN254 (pairing-based zk-SNARK)	trusted	no
3	LAS (simplified Dilithium-III)	LaZer (LNP22 linear relation with norms)	transparent	yes

to make it one: both LAS configurations share one expansion of the public matrix, prove the *identical* relation $\exists\, r' : Ar' = t' \wedge \|r'\|_\infty \leq 1$ — the hard relation of the statement/witness pair, *not* the signer's key pair, including the norm bound, since proving only $Ar' = t'$ would fail to close the knowledge gap — and derive all key material from one pinned seed. This is verified rather than assumed: each scheme and proof system reports a *relation binding*, a digest computed by evaluating the scheme's own linear map on fixed probe vectors, and a configuration whose halves disagree refuses to run. Deriving that binding through the scheme's own code rather than from the seed is deliberate — a seed would certify only that both halves were *asked* for the same parameters, not that both compute with them. The $1 \to 2$ step is not controlled and is not presented as though it were: it changes the signature scheme, the relation and whether a role-A proof exists at all, so it is reported as a whole-stack comparison, the signature-only question being answered by section 3.4.1.

The Groth16 circuit. No pairing-based proof of this lattice statement was available to reuse, so the circuit is new. It is far cheaper than the statement suggests, because $r' \mapsto Ar'$ is *linear with public coefficients* and so needs no witness–witness multiplication, the expensive ingredient in a rank-one constraint system: the public linear map is enforced through linear constraints, and the only non-linear and range constraints are the ternary bound and the reduction modulo q. The constraint count is therefore linear in the dimensions rather than in their product. Appendix A.3 gives the construction and the recovery of the matrix from the scheme's own linear map.

Measurement. Because gas does not exist on this venue, the axes are execution time and communication cost, the latter counting *off-chain* protocol messages and not only what reaches a ledger. Sizes are observed from the transmitted buffers rather than computed from a formula, so these timings sit at the **packed tier** and are not comparable with core-tier figures elsewhere. Each configuration runs 10 complete swaps, reported as mean ± sample standard deviation, with one-time costs excluded and reported separately. Appendix A.3 sets out the invariants enforced on phase timing and communication subtotals, the dedicated batch the rejection gate is applied to here, and the one deliberate exception to reproducibility: Groth16's setup and proofs draw fresh entropy for security reasons, leaving proof size unaffected but adding a little variance to proving time.

Chapter 3

Results and discussion

This chapter reports what was measured and why the results should be believed: correctness first, then computation cost, then communication cost with its component breakdown, and finally the application results.

3.1 Correctness and robustness

No performance claim means anything until the implementation is shown *correct*, and an adaptor signature is not correct merely because it signs and verifies — it must satisfy a specific contract. Table 3.1 lists every clause and its outcome; all pass, with zero compiler warnings under the project's strict flags. Clauses 1–4 are the adaptor contract itself: a pre-signature is verifiable yet unspendable (the tripwire), becomes an ordinary signature once the witness is added, and completing it reveals that witness — the mechanism that makes an atomic swap atomic. Clauses 5–6 are robustness, and clause 7 the reproducibility that lets an independent verifier be cross-checked against fixed vectors.

3.2 Computation cost

The primary computation result is the cost of the *adaptor layer* — what LAS adds to the base signature it is built on. The two paths share algorithm, parameters and primitives, so any difference is purely the price of adaptor functionality (section 2.6). Table 3.2 reports it at *both* measurement boundaries and fig. 3.1 visualises the core tier. At the core boundary, at the target setting, every adaptor operation stays within single digits of its counterpart and Extract — which has

Table 3.1: The adaptor-signature correctness contract, each clause checked and passed by `test_contract` (corroborated at scale by the 1000-iteration functional suite on modes 2/3/5, the 256-instance serialization suite, and the pinned known-answer test).

#	Property	Result
1	PreSign produces a pre-signature that PreVerify accepts	pass
2	the pre-signature *fails* basic Verify (statement-binding tripwire)	pass
3	Adapt yields a signature that basic Verify accepts	pass
4	Extract recovers the witness *exactly* ($y' = y$)	pass
5a	a tampered message fails Verify	pass
5b	every one of the 4640 single-byte flips of the packed signature is rejected	pass
6	malformed packed bytes are rejected ($pk \geq Q$, ternary code-3, z out of band)	pass
7	the deterministic API is byte-identical on re-run	pass

no base analogue — is the cheapest of all; every operation completes in well under two milliseconds, signing and pre-signing dominating because of rejection sampling.

At the core boundary, then, the adaptor layer is almost free in computation, for structural reasons: PreSign (+1.6% over Sign) runs the same reject loop, adding only Y to the commitment and a tighter bound; PreVerify (+5.1% over Verify) recomputes the same commitment shape with the extra $+Y$; Adapt (+9.0% over Verify) runs a PreVerify then adds the witness vector, costing about one verification plus a few polynomial additions; and Extract merely subtracts $z - \hat{z}$ and checks $A\,y = Y$. None changes the asymptotic work. This is the headline of the standalone-signature evaluation: *at the core tier*, adaptor functionality costs only single-digit percentages of extra computation, and the two operations unique to an adaptor signature cost no more than a verification each. The premium stays in single digits at all four parameter sets, so it is not an artefact of one choice (fig. A.1, appendix).

That argument concerns the core tier alone. The packed-tier block of table 3.2 shows what changes when the boundary moves to packed bytes, as a wire or on-chain verifier requires: the premium grows to +20.8%, +35.3% and +79.5% — an order of magnitude above the core-tier figures, and no longer single-digit. This gap is serialization, not adaptor arithmetic: each adaptor operation must

Table 3.2: The primary computation result: per-operation adaptor overhead at the target setting *Simplified Dilithium-III* (n=6, ℓ=5, κ=49, γ=137 984, d=256), at *both* measurement boundaries of section 2.6. Each LAS adaptor operation is paired with the basic operation it mirrors; "Overhead" is the extra cost as a percentage of that basic operation, within the same tier. All timings are measured on the C implementation, the implementation of record (section 2.6); the Rust port's independent measurements are in table 3.4. Timings are μs per operation (mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500/1000 iterations; machine of record in section 2.6). KeyGen, Sign, and Verify are reused unchanged by LAS; Extract has no basic analogue. The packed-tier overheads are larger because each adaptor operation additionally decodes the public-key-sized statement Y (4416 B) — serialization, not adaptor arithmetic. Absolute core-tier timings at every parameter set are in table A.1 (appendix).

Operation (basic / adaptor)	Basic (μs)	LAS adaptor (μs)	Overhead
Core tier (structures in/out — isolates the adaptor arithmetic)			
KeyGen	91.6 ± 2.0	91.6 ± 2.0 (shared)	—
Sign / PreSign	852 ± 25	866 ± 41	+1.6%
Verify / PreVerify	199 ± 6	210 ± 9	+5.1%
Verify / Adapt	199 ± 6	217 ± 5	+9.0%
Extract	—	84.2 ± 6.3	(LAS only)
Packed tier (wire bytes in/out — incl. validating decode + encode)			
KeyGen	314 ± 5	314 ± 5 (shared)	—
Sign / PreSign	1384 ± 37	1673 ± 49	+20.8%
Verify / PreVerify	703 ± 37	951 ± 18	+35.3%
Verify / Adapt	703 ± 37	1261 ± 20	+79.5%
Extract	—	940 ± 43	(LAS only)

additionally decode the public-key-sized statement Y (4416 B), and Adapt repacks the completed signature. The object that dominates the *communication* cost (section 3.4) therefore reappears as the dominant *computation* cost once the byte boundary is included, so a deployment verifying from bytes should budget for the packed row. Presenting both tiers with each boundary stated is deliberate: reported lattice-signature timings are notoriously sensitive to where the boundary is drawn [15].

Signing and pre-signing are dearer than verification because of rejection sampling, which is intrinsic to the Fiat–Shamir-with-aborts family rather than a defect here. Equation (2.2) predicts 2.719 attempts per Sign and 2.775 per PreSign, the latter restarting about 2% more often because its bound is tighter by exactly one; measured directly, by counting attempts rather than estimating from a timing

Table 3.3: Rejection-sampling statistics at the target setting *Simplified Dilithium-III*: the closed-form geometric model of eq. (2.2) against every measured sample. "Mean" is attempts per call and "Accept." is the per-attempt acceptance rate — the two quantities every source records, and the ones the model predicts. The per-call *dispersion* (the geometric standard deviation $\sigma = E\sqrt{1 - 1/E}$, the medians and 95th percentiles, and the sampled maxima) is annotated on the companion acceptance curve of fig. 3.2 rather than repeated here. The C driver is the implementation of record; the Rust protocol driver and the Criterion harness are the independent samples of section 2.5. Every measured row passes the five-standard-error validity gate of section 2.6.

Operation	Source (calls)	Mean	Accept.
Sign	geometric model, eq. (2.2)	2.719	36.8%
	C driver, distribution sample (2000)	2.715	36.8%
	C driver, timed-run gate (2500)	2.677	37.4%
	Rust driver, timed-run gate (2500)	2.677	37.4%
	Rust Criterion, gate (94395)	2.725	36.7%
PreSign	geometric model, eq. (2.2)	2.775	36.0%
	C driver, distribution sample (2000)	2.716	36.8%
	C driver, timed-run gate (2500)	2.714	36.9%
	Rust driver, timed-run gate (2500)	2.765	36.2%
	Rust Criterion, gate (94395)	2.777	36.0%

ratio, the run of record observed 2.715 and 2.716 respectively (table 3.3), and fig. 3.2 shows how quickly acceptance accumulates. At the drivers' 2500 timed calls the gate band ($\pm\sim 8\%$) is wider than that 2% separation, so the two counts are statistically indistinguishable at this sample size — which is why the core-tier PreSign overhead (+1.6%) should be read as of the same order as the attempt-count prediction, not as a precise constant. Only the $\approx 10^5$-call Criterion harness, band $\pm\sim 1\%$, resolves it.

3.3 Cross-implementation check: C implementation versus Rust port

The scheme exists twice (section 2.5), and the two implementations agree at every level at which they can be compared. At the byte level agreement is exact: the same packed sizes at the target setting (public key 4416 B, signature 6720 B, checked programmatically on every Rust run) and the same pinned known-answer digest (`bb6ad0da...260c`). Since that digest covers key generation, signing, the four adaptor operations and the serialization, a divergence anywhere in either implementation's sampling, algebra or encoding would flip it; its equality is a strong, cheap argument that the two codebases compute the same scheme.

Timing agreement is necessarily looser, the builds going through different compilers, but close: table 3.4 shows every operation agreeing within about 17% (largest gap KeyGen). More importantly the central conclusion reproduces in both languages. At the core tier the Rust port measures +1.1%, +0.5% and −4.9% for PreSign, PreVerify and Adapt against their base operations, all single-digit in magnitude alongside the C values of +1.6%, +5.1% and +9.0%. The Rust Adapt figure is slightly *negative*: at this scale the genuine overhead is smaller than the cost gap between the base and LAS verify paths, separate helpers compiling to marginally different code, so the measurement floor rather than a real speed-up fixes its sign. The packed tier reproduces too — +1.2%, +12.2% and +27.1%, all positive and in the C figures' order — confirming that once statement decoding is included the adaptor premium is a cross-language signal, not a compiler artefact. Both ports' counted rejection rates pass the same gate, and the Criterion harness gives lower absolute times but the same ordering and shape (fig. 3.3). The headline results are therefore properties of the algorithm, not of one codebase, compiler or harness.

Table 3.4: Per-operation timing of the two implementations at the target setting, *Simplified Dilithium-III* (μs per operation). The C and Rust columns come from the two protocol drivers, which deliberately mirror each other (same repetition scheme of 5 repetitions $\times$ 500/1000 iterations, fixed public-parameter seed, fixed message, same success-path gating), so they are directly comparable. The final column is the independent Criterion.rs statistical harness (300 samples per operation, bootstrap 95% confidence interval of the mean); its hot-loop protocol yields lower absolute times, and it is included to show that the *relative* conclusions do not depend on the hand-rolled driver.

Operation	C implementation (μs)	Rust port (μs)	Rust / C	Rust, Criterion (μs, 95% CI)
KeyGen	91.6 ± 2.0	108 ± 8	1.17	84.4 [83.8, 85.0]
Sign	852 ± 25	894 ± 68	1.05	751 [745, 757]
Verify	199 ± 6	203 ± 18	1.02	152 [151, 153]
PreSign	866 ± 41	903 ± 28	1.04	773 [767, 779]
PreVerify	210 ± 9	204 ± 17	0.97	153 [152, 154]
Adapt	217 ± 5	193 ± 10	0.89	156 [155, 158]
Extract	84.2 ± 6.3	71.7 ± 1.3	0.85	57.4 [56.9, 58.0]

3.4 Communication cost and component breakdown

The computation results show the adaptor layer is cheap at the core boundary; the size results show where the real cost lies. Table 3.5 decomposes every LAS object into bytes at the target setting, to explain *which* component drives the size rather than merely state that signatures are large. Sizes are fixed by the encoding, so they are reported once, as exact bytes with no dispersion.

Two points follow. First, **the response z is essentially the entire signature** — 99.3% at Simplified Dilithium-II, rising to 99.7% at -V — since the challenge travels as the fixed 32-byte hash $\tilde{c}$ (section 2.3); any optimisation of LAS signatures is therefore an optimisation of z. Second, **an adaptor swap additionally transmits a public-key-sized statement Y and a signature-sized pre-signature**, the witness being small; these are the only objects an adaptor protocol sends that a basic signature does not.

The sharp conclusion: the cost of LAS is not adaptor computation but communication — specifically the response vector z and the statement Y. At the core tier the adaptor operations add at most +10.5% of compute, while the objects moving on the wire are one to a few kilobytes, dominated by z. That

Table 3.5: Serialized size of every LAS object at the *Simplified Dilithium-III* (target) setting, in packed wire bytes (not on-chain gas), with its paper notation. "In the basic signature?" marks which objects LAS shares with the base scheme and which it adds. Three facts read off directly: the signature, pre-signature, and adapted signature are byte-identical (adaptation changes the *value* of the response, not its size); the response z dominates every signature; and the only extra *public* object LAS adds over a basic signature is the statement Y, the size of a public key. The sizes hold for the C and the Rust implementation alike: the wire encoding is byte-identical in both by construction, asserted programmatically on every Rust run (section 3.3). Sizes at every parameter set: table A.2 (appendix).

Object	Notation	Bytes	In the basic signature?
Public key	$pk = t$	4416	yes (shared)
Secret key	$sk = r$	704	yes (shared)
Challenge	c	32	yes
Response	z / $\hat{z}$	6688	yes
Signature	(c, z)	6720	yes (basic signature)
Statement	$Y = t'$	4416	LAS only (public)
Witness	$y = r'$	704	LAS only (private)
Pre-signature	$(c, \hat{z})$	6720	LAS only
Adapted signature	(c, z)	6720	LAS (verifies as basic sig)

matters more in a blockchain setting than "the signature is 6720 bytes": bytes reaching a ledger are stored and replicated by every node, while the statement and pre-signature, exchanged off-chain, still cost both parties bandwidth. The response is stored at full width — 18–19 bits per coefficient, neither range-reduced nor entropy-coded — the clearest opportunity for reducing that footprint (section 5.3).

3.4.1 The price of post-quantum: classical adaptor baseline

The second baseline is a classical secp256k1 ECDSA adaptor signature [2]: the natural functional counterpart of LAS, with the same operation set and driving the same scriptless swaps, but resting on classical discrete-log rather than lattice hardness. It therefore measures the practical cost of moving from a classical adaptor signature to a post-quantum one. LAS is instantiated at *Simplified Dilithium-II* and only there: secp256k1 offers roughly 128-bit classical security and the nearest post-quantum target is NIST level 2, so those dimensions are the most like-for-like engineering match, while the -III or -V settings would pit the classical scheme against a deliberately stronger configuration. Following the tier rule, LAS appears at both its boundaries while the classical scheme sits at the

hybrid native-API boundary (table 2.3), so the comparison is read conservatively — against LAS's core tier it slightly flatters LAS, against its packed tier the classical scheme — but the conclusions survive either way, resting on order-of-magnitude gaps rather than percent-level differences.

Two findings stand out. First, **the post-quantum price is paid almost entirely in size**: LAS's signature and public key are roughly 72× and 89× larger than the classical adaptor's, whereas in computation every LAS operation stays well under a millisecond — even the largest per-operation factor (Adapt, about 52×, against a classical operation that is nearly free) costs under a tenth of a millisecond absolutely. Second, **the two schemes pay their adaptor overhead in opposite ways**: the classical adaptor must attach a discrete-logarithm-equality proof [7], making its PreSign and PreVerify several times its own base sign and verify, whereas LAS folds the statement into a hash it already computes and so stays within a few percent of its base at the core tier. The lattice adaptor layer is thus relatively cheaper to add, and the post-quantum cost surfaces in communication — consistent with the component analysis above.

3.5 Application demonstration

The implementation drives an end-to-end post-quantum atomic swap following Fig. 1 of [6] *verbatim*, including its proof of knowledge π, which appendix A.6 sets out step by step together with the proof's construction over the LaZer library [10] and its test suite. Two results matter here. First, the protocol runs and every step is asserted, including the counterfactuals that a pre-adaptation signature fails basic verification and that π is rejected against any other statement; a modelled ledger additionally carries a timeout/refund path and a multi-hop payment. Second, π is what makes the second party's final step *guaranteed* rather than merely expected: by the Module-SIS uniqueness argument of [6] the extracted witness equals the proven ternary one, so the adapted signature is certain to clear the verification bound — without π a malicious counterparty could claim and leave an unadaptable pre-signature behind. The proof measures approximately 31 kilobytes at a knowledge error below 2^{-127}, and is exchanged off-chain only, exactly as [6] note, so it adds nothing to on-chain cost.

As a supporting check on deployability, native on-chain LAS verification was *implemented* and measured on a real Solidity stack at the target parameter

set: a complete, numerically-correct byte-level verifier assembled from vendored ZKNox ETHDILITHIUM primitives and validated end-to-end against the C implementation, which it matches on a valid signature and on tampered bytes. Wired into the swap, one full verified settlement measured approximately 56.5 million gas (table 3.7; fig. 3.4) — roughly 746× the equivalent classical ECDSA-adaptor claim, which verifies in a native precompile whereas LAS runs entirely in EVM bytecode. This supersedes an earlier op-count *estimate* of approximately 16.7 million gas, the measured figure being larger because a complete verifier must also decode the signature, run a real Solidity SHAKE256 and canonically pack its inputs. At that price it exceeds the EIP-7825 per-transaction cap of 16 777 216 gas; raw demand sits below the 60-million default block limit, but the per-transaction cap binds. An *optimistic* (Naysayer) alternative was therefore also implemented and measured, with a deliberately mixed result: honest settlement becomes cheap (approximately 1.1 million gas per claim), but the worst-case fraud proof — a Solidity SHAKE256 hash dispute at approximately 29.4 million gas — still exceeds the cap on execution gas alone (appendix A.7). Both on-chain results are supporting evidence: they inform, but do not displace, the first-stage signature results above.

3.6 The swap on a UTXO chain: three configurations compared

The demonstration above establishes that the protocol runs. This section measures what it *costs* and isolates which component is responsible, over the UTXO ledger and under the design of section 2.7: three configurations (table 2.6), of which only the $2 \to 3$ step is controlled. Timings are the mean ± sample standard deviation over 10 complete swaps at the *packed* tier, so they are not comparable with the core-tier figures of section 3.2. Both LAS configurations passed the rejection gate, measuring 2.8655 attempts per PreSign against the closed-form 2.77483.

The proof, not the signature, is the cost of a post-quantum swap. The role-A proof consumes 99.2% of end-to-end time in configuration 2 and 98.3% in configuration 3. Strip it out and the entire post-quantum signature workload — two key generations, a statement, two pre-signatures, a pre-verification, two adaptations, an extraction and two settlements — costs 8976.0 μs, about 5.1×

the classical swap's 1754.2 μs. A factor of five on a millisecond-scale workload is unremarkable; a proof of hundreds of milliseconds decides whether the protocol feels instantaneous. Even at the packed tier the signature workload is not the bottleneck — the post-quantum penalty is not where intuition puts it.

The controlled comparison inverts the expected trade-off. Replacing Groth16 with LaZer — same signature, same **A**, same inputs — makes the swap *faster* by 53.0% while making it *larger* by 61.9%: the post-quantum system is roughly twice as quick to produce a proof yet emits one about 240× bigger (128 B against 30717–30766 B). Two structural facts drive this. Groth16 is *succinct* — three group elements whatever the circuit — but producing that proof means a multi-scalar multiplication over the whole constraint system, whereas LaZer is not succinct, its proof growing with the statement, but proving a lattice relation with lattice machinery avoids encoding arithmetic over a 23-bit modulus into a 254-bit pairing-friendly field. Verification runs the other way: Groth16 verifies in 29365.8 μs against LaZer's 164155.4, succinctness paying off exactly where it is designed to.

Configuration 2's trade is worse than it looks. Pairing security is broken by Shor, so its proof is precisely the component a post-quantum adversary attacks: a post-quantum signature does not protect a swap whose fairness argument rests on a classical proof. Groth16 also requires a *trusted, per-circuit* setup — 2.753739356s here — whose secret must be destroyed for soundness, whereas LaZer's parameters are public and transparent. Configuration 3 is thus not merely the post-quantum option: it is faster end-to-end, needs no trusted party and removes a quantum vulnerability, paying only in bytes.

Communication is again the real price, with a usability consequence. A classical swap moves 941 B in total; the fully post-quantum swap moves 80001–80050 B, about 85.0× more, and puts 63.0× more on the ledgers. That on-chain multiple is what matters economically, since a UTXO chain prices transactions by size — and it is also why the venue was changed, the same objects costing prohibitive gas on the EVM (table 3.7). In practical terms a fully post-quantum swap needs 517799.7 μs of computation and tens of kilobytes of off-chain messaging per exchange, overwhelmingly the proof, against the classical swap's 1754.2 μs: comfortable on a laptop and plausible on a modern phone, but a large enough

gap that a naive port of a mobile wallet's UX assumptions would mislead. No mobile or low-power device was measured, so that point should not be over-read (section 3.7).

3.7 Threats to validity

Several factors qualify these results. All timings are machine-dependent, mitigated by reporting standard deviations, fixing one machine and compiler, emphasising ratios over absolutes, and by the cross-implementation check (section 3.3). The LAS parameter set's concrete security level is not formally established, security proofs being out of scope, so every cross-scheme claim is qualified by the parameters of table 2.4; the modulus is $q \approx 2^{23}$ rather than the 2^{24} of [6], which leaves correctness unaffected ($q > 2\gamma$) but changes the security margin. The extracted witness is exact here, whereas the relaxed setting of [6] allows witness noise that can grow across long payment-channel paths; exact extraction sidesteps that growth but is a simplification.

Two further factors are specific to section 3.6. The UTXO ledger is an in-process model, not a consensus implementation — no peer-to-peer layer, mempool policy, fee market, reorganisations or script interpreter — so it yields real transaction objects, real serialised sizes and the real leak channel on which atomicity depends, but no fees or confirmation latency; on-chain byte counts are the honest proxy and are reported as such. This is a property of the venue rather than a shortcut: running configurations 2 and 3 against a real Bitcoin node is impossible without a consensus change, which is exactly why [6] states its application over a UTXO chain *whose signature algorithm has been replaced*. Finally, the Stage-2 timings come from a single desktop machine (AMD Ryzen 7 7745HX with Radeon Graphics), so the usability discussion above is an extrapolation, not a measurement on constrained hardware.

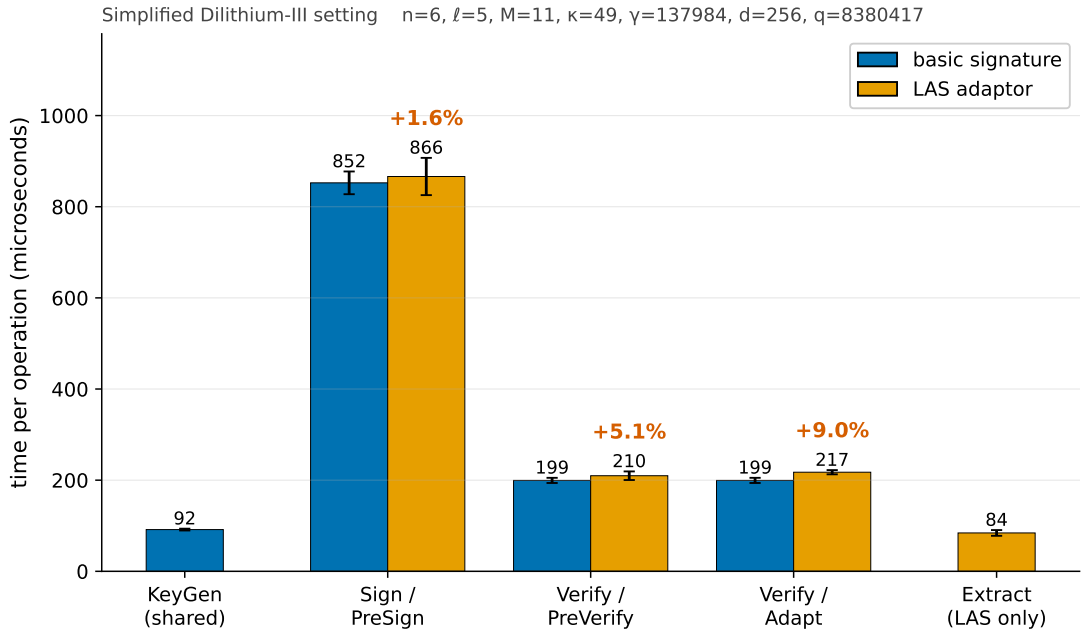

Figure 3.1: The primary computation result, shown visually: each LAS adaptor operation (orange) paired beside the basic operation it mirrors (blue), at the target setting *Simplified Dilithium-III* (its parameters are annotated above the plot), measured on the C implementation (the implementation of record, section 2.6). The percentage above each orange bar is the adaptor overhead over its blue partner; exact means and standard deviations, and the packed tier, are in table 3.2. KeyGen, Sign, and Verify are reused unchanged by LAS, so KeyGen is shown once (shared) and Sign/Verify are the blue partners of PreSign/PreVerify/Adapt; Extract has no basic analogue and is shown LAS-only. The absolute μs scale also shows the shape: every operation is sub-two-milliseconds and the rejection-sampling operations (Sign, PreSign) dominate, while the verify-class operations and Extract are far cheaper. Error bars are the sample standard deviation over 5 repetitions of 500 (sign-class) / 1000 (verify-class) iterations on the machine of record (section 2.6); the same comparison across all four parameter sets is the scaling check in fig. A.1 (appendix).

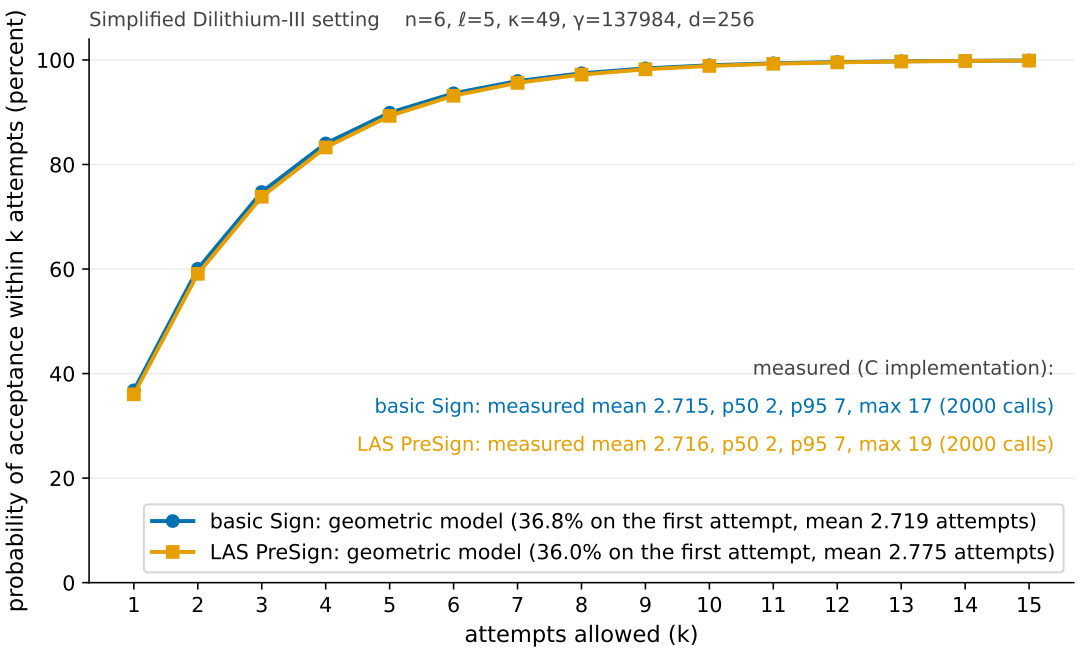

Figure 3.2: The probability that a sign-class call has been accepted *within* k rejection-sampling attempts, at the target setting *Simplified Dilithium-III*. The curves are the closed-form geometric model of eq. (2.2) for the basic Sign bound $\gamma - \kappa$ (blue) and the tighter LAS PreSign bound $\gamma - \kappa - 1$ (orange): acceptance is 36.8% and 36.0% respectively on the first attempt, passes 90% by five attempts, and then flattens — the two curves are nearly coincident, which is exactly why the PreSign-versus-Sign timing gap is so small. The annotated statistics are *measured* on the C implementation (the 2000-call distribution sample tabulated exactly in table 3.3). The flattening is the practical point: allowing many more attempts buys almost nothing, because the overwhelming majority of calls are accepted within the first few.

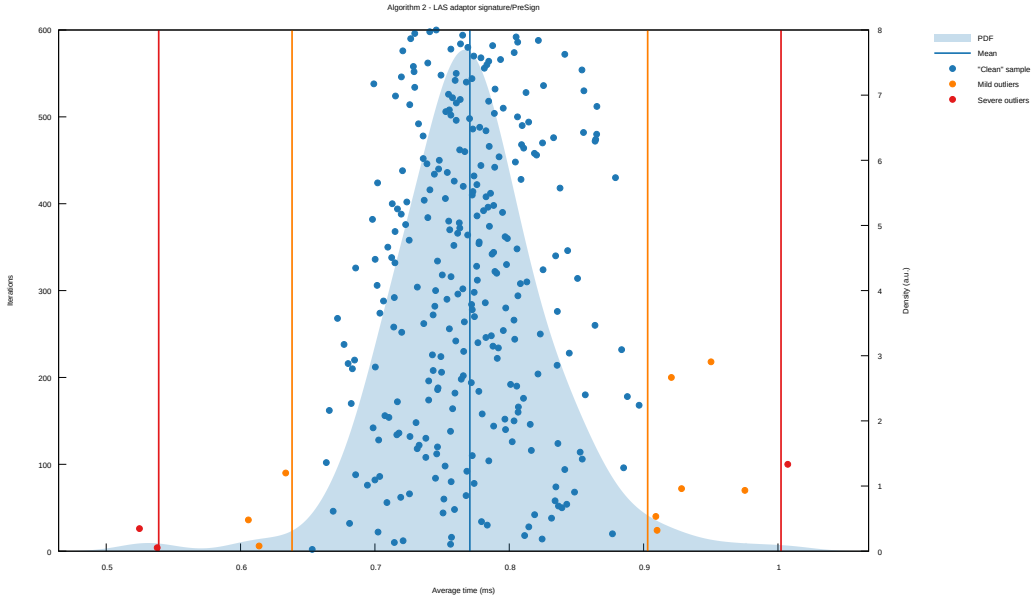

Figure 3.3: The statistical harness's own report for PreSign at the target setting, reproduced unmodified from Criterion.rs (its SVG output converted to PDF): 300 timed samples (dots), the kernel-density estimate of the sampling distribution (shaded), the mean (vertical blue line), and Criterion's automatic outlier classification (mild / severe, orange / red). This is the evidence behind the Criterion column of table 3.4: a well-formed, single-mode distribution whose spread comes from rejection sampling and scheduling noise, with only a handful of classified outliers.

Table 3.6: Classical vs. post-quantum adaptor signature: same functionality, different security foundation. Classical secp256k1 (≈128-bit *classical* security) versus LAS at the *Simplified Dilithium-II* parameters ($n = 4$, $\ell = 4$, $M = 8$, $\kappa = 39$, $\gamma = 79\,872$, $d = 256$, $q = 8\,380\,417$). The Dilithium-II dimensions are an engineering match to the ≈128-bit (NIST level 2) target, *not* a formal security-equivalence claim (table 2.4); the stronger LAS settings are reserved for the parameter-sensitivity study (section 3.2) and would overstate the post-quantum cost here. Timings μs/op (classical: mean ± SD over 10 runs × 1000 iterations; LAS: 5 repetitions × 500/1000 iterations; same machine); sizes in bytes. Measurement boundaries (section 2.6): the classical column is the library's *hybrid* native-API boundary — PreSign/PreVerify/Adapt/Extract pack or parse the 162-byte pre-signature *inside* the timed calls (packed-like), while KeyGen/Sign/Verify exchange in-memory structs with their wire codecs outside (core-like); LAS is shown at both its core and packed tiers, measured on the C implementation. KeyGen also produces the statement/witness, which take the public-key/secret-key size. The classical pre-signature is a syntactically distinct object (an ECDSA signature plus a discrete-logarithm-equality proof), whereas the LAS pre-signature shares the signature format.

	ECDSA adaptor (classical, hybrid native API)	LAS adaptor (post-quantum, core tier)	LAS adaptor (post-quantum, packed tier)
Computation (μs/op)			
KeyGen	27.2 ± 1.9	61.7 ± 0.8	221 ± 15
Sign	36.1 ± 1.0	570 ± 20	893 ± 27
Verify	58.1 ± 7.8	131 ± 3	326 ± 45
PreSign	171 ± 10	608 ± 35	1136 ± 56
PreVerify	219 ± 7	135 ± 2	590 ± 14
Adapt	2.8 ± 0.1	144 ± 7	835 ± 19
Extract	30.6 ± 2.5	55.6 ± 4.2	641 ± 12
Communication (bytes)			
Public key / statement	33	2944	
Secret key / witness	32	512	
Signature	64	4640	
Pre-signature	162	4640	

Table 3.7: On-chain settlement gas for the atomic swap's claim step, measured on a local EVM (Foundry `forge -gas-report`). Gas is deterministic for the fixed bytecode, EVM revision, inputs, and contract state, and is independent of the host CPU. The ratios compare complete settlement calls, including calldata, memory, state transition, event emission, transfer, and verification. Both the classical path and the full LAS path perform *complete* signature verification — the difference is that ECDSA verifies in the native `ecrecover` precompile whereas LAS runs its entire verifier in EVM bytecode. `claimLAS` is a floor that performs *no* lattice verification (calldata for the 6720-byte signature plus one `keccak256`), a strict lower bound. `claimLASVerified` runs the complete native `base_verify` (section 2.1) — decode, the norm bound, $w' = A\mathbf{z} - c\,t$, and the SHAKE256 challenge check — assembled from the vendored ZKNox ETHDILITHIUM primitives and validated end-to-end against the C reference. At ≈56.5 M gas it exceeds the EIP-7825 per-transaction cap ($16\,777\,216 = 2^{24}$) by ≈3.4×, so it cannot execute as a single mainnet transaction; its raw demand is below the 60-million default block gas limit, but the per-transaction cap is the binding ceiling.

Claim path	On-chain verification	Gas	× classical
Classical ECDSA adaptor (`claimClassical`)	full, `ecrecover` precompile	75 751	1×
LAS floor (`claimLAS`)	none (calldata + `keccak256`)	289 930	3.8×
LAS full verify (`claimLASVerified`)	full `base_verify`, in Solidity	56 538 682	746×

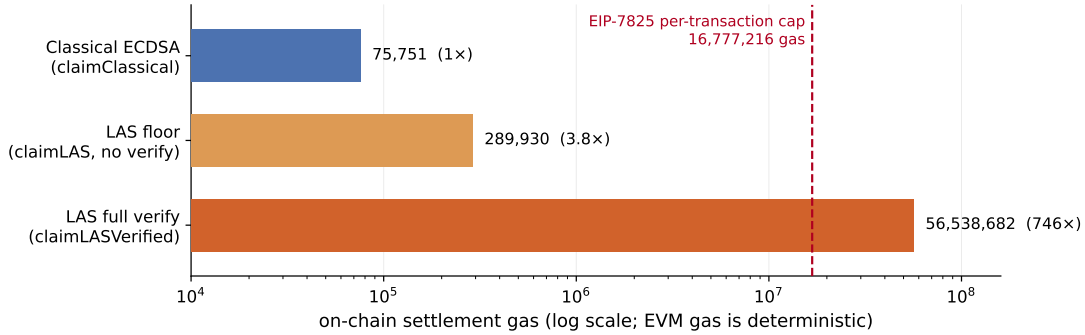

Figure 3.4: On-chain settlement gas for the three atomic-swap claim paths (table 3.7), measured on a local EVM (`forge -gas-report`; deterministic EVM gas), on a logarithmic axis. The classical `claimClassical` (75 751 gas) verifies in the native `ecrecover` precompile; `claimLAS` (289 930) is a settlement floor doing *no* lattice verification; and `claimLASVerified` (56 538 682, ≈746× the classical claim) runs the complete native LAS `base_verify` in EVM bytecode. The dashed line is the EIP-7825 per-transaction gas cap ($16\,777\,216 = 2^{24}$); the full verifier's bar crosses it, which is exactly why it cannot execute as a single mainnet transaction and why a deployable path needs a precompile, a succinct proof, or an optimistic (Naysayer) scheme.

Table 3.8: Per-phase execution time of one complete Fig. 1 swap, mean ± sample SD over 10 swaps (μs; run 20260725_202359, AMD Ryzen 7 7745HX with Radeon Graphics). Dashes mark phases a configuration does not have. End-to-end is context, not the headline.

Phase	Classical	LAS + Groth16	LAS + LaZer
KeyGen ×2 + Gen	127.0	1140.5	1127.0
Prove (π)	—	1064348.9	344633.7
PreSign (u_1)	244.9	2295.0	2289.6
ProofVerify (π)	—	29365.8	164155.4
PreVerify (abort gate)	309.5	808.2	800.6
PreSign (u_2)	245.4	1668.4	1706.4
Adapt (u_1)	308.8	800.5	805.4
Settle chain 2	91.3	526.8	530.9
Extract (u_2)	51.1	418.9	424.9
Adapt (u_2)	290.1	762.5	788.4
Settle chain 1	86.0	555.3	537.5
end-to-end	1754.2	1102690.8	517799.7
of which π	—	99.2%	98.3%
excluding π	1754.2	8976.0	9010.7

Table 3.9: Communication cost of one complete swap, in bytes, observed from the transmitted buffers over 10 swaps. A range indicates a message whose size varied between runs. Off-chain messages are counted, not only what reaches a ledger.

Message	Classical	LAS + Groth16	LAS + LaZer
statement Y	33	4416	4416
proof π	—	128	30717–30766
$\hat{\sigma}_1$	162	6720	6720
tx_1	114	4497	4497
$\hat{\sigma}_2$	162	6720	6720
tx_2	114	4497	4497
off-chain subtotal	585	26978	57567–57616
on-chain subtotal	356	22434	22434
total per swap	941	49412	80001–80050

Chapter 4

Evaluation

This chapter judges the work against the objectives of section 1.3. It first justifies the evaluation strategy and its alignment with those objectives, then assesses each objective against the evidence of chapter 3, then states the implementation challenges that shaped the outcome and the limitations that bound it. Judgement of *what the artefact achieves* is made here; section 5.2 steps back from the artefact to the process, reflecting on the strategy, on what a second attempt would change, and on the shortfalls that outlive the measurements.

4.1 Evaluation strategy and its justification

The aim was to establish what adaptor functionality *costs*, so the evaluation had to make each measured difference attributable to one cause. Four decisions follow from that, each tied to an objective.

Correctness is established before any cost is reported (objective 3). A fast but wrong scheme measures nothing, and no formal proof was in scope, so correctness rests on differential evidence: a pinned known-answer digest reproduced byte-for-byte by a second, independently written implementation (section 3.3). Internal self-consistency is not correctness — section 4.3 gives a case where every primitive passed in isolation and the assembled whole was still wrong — so a value shared across two implementations is treated as the strongest implementation-level check available here.

The primary comparison is controlled, not merely convenient (objective 4). LAS is measured against its own simplified base at identical algorithm, parameters and primitives, so the difference between the two paths is the adaptor

layer and nothing else. Comparing instead against the optimised CRYSTALS-Dilithium reference would confound the adaptor cost with parameter, packing and hint-vector differences; that comparison is therefore retained only as context, and the classical ECDSA adaptor only as a functionality-matched "price of post-quantum" baseline.

Every measurement declares its boundary. Results are reported at two tiers, core and packed (section 2.6), because an undeclared boundary is the commonest way a benchmark misleads — as it did twice here before the tiers were fixed (section 4.3). For the same reason each run gates on a directly counted rejection rate matched against its closed-form prediction, rather than inferring acceptance from timing.

Metrics follow the venue (objective 5). Stage 1 reports the five standard signature metrics. The application study reports execution time and communication bytes with off-chain messages counted, because the evaluated UTXO ledger has no gas metric to report — and because off-chain messaging is where the post-quantum cost actually lands.

4.2 Achievement of the objectives

Table 4.1 states the verdict on each objective and the evidence for it. All five were met; two carry qualifications that matter.

Objective 4's fairness claim rests on control rather than assertion: in Stage 1 the two compared paths share algorithm, parameters and primitives, and in Stage 2 the two LAS configurations share one public matrix and receive byte-identical inputs, verified at run time, so the difference between them is attributable to the proof system alone. The measurements are then defended from two independent directions — a closed-form gate on the rejection rate, and re-measurement under a different compiler and an independent statistical harness.

Objective 5 was met, with evidence extending beyond the original demonstration, but only after the first venue failed. On a local EVM a *complete*, validated native LAS verifier was measured at approximately 56.5 million gas (table 3.7), above the EIP-7825 per-transaction cap, and an optimistic (Naysayer) variant then made honest settlement cheap while leaving a worst-case fraud proof that also exceeds it. That negative result is quantified rather than assumed, and it motivated rebuilding the protocol over a UTXO ledger of the kind [6] assumes.

Table 4.1: Each objective of section 1.3 judged against the evidence reported in chapter 3.

Objective	**Evidence**	**Verdict**
1. Literature and scheme selection	Adaptor signature selected as the exotic scheme of highest blockchain value; LAS [6] selected as the design with the shortest path from a standardised lattice signature (section 1.2).	Met
2. Additive implementation	Lattice core reused byte-for-byte, zero upstream functions modified, made auditable by a two-branch diff; adaptor layer, wire encoding and application are new code (table 2.1).	Met
3. Validation	Functional, serialization, tamper and known-answer tests pass (section 3.1); an independent Rust implementation reproduces the wire format and the pinned digest byte-for-byte (section 3.3).	Met
4. Fair benchmarking	Adaptor overhead isolated per operation at two declared tiers (table 3.2); two further baselines, five standard metrics, computation separated from communication; rejection rate gated against theory; relative conclusions reproduced by the Rust port (table 3.4).	Met
5. Atomic-swap demonstration	End-to-end two-party, two-chain swap settling both legs and asserting unspendability before adaptation (section 3.5); extended to a measured three-configuration study on a UTXO ledger (section 3.6).	Met

The two together — why the smart-contract venue does not work, and what the alternative costs — are stronger than either alone. Deployment on a live network remains out of scope.

4.3 Implementation challenges

Turning the simplified-Dilithium base into LAS was conceptually small — four added operations and one extra term in a hash — but the six problems of table 4.2 consumed most of the engineering effort, and they explain how the results were produced. Three recur as themes. Two of the six are failures that *pass every local test*: a loosened PreSign bound leaves the adaptor operations working while breaking ordinary verification two steps later, and a doubly-transformed public matrix produces a verifier whose parts agree with each other but not with the C implementation. Both were caught only by checking against something outside the component — a norm-budget argument in the first case, a cross-implementation digest in the second — which is why section 4.1 treats internal self-consistency as insufficient. Two more are measurement artefacts that would have biased the headline result had they survived, and are why every boundary is now declared and every run gated. The last is asymptotic rather than arithmetic, and generalises past this project: in constraint-system libraries the ergonomic interface and the scalable interface are not the same interface.

4.4 Limitations

The validity threats of section 3.7 bound what the results mean and are not repeated here. Four are worth restating as limitations of the artefact rather than of the measurement: the parameter set's concrete security is unanalysed by design; the modulus is $q \approx 2^{23}$ rather than the 2^{24} of [6]; the witness is extracted exactly, sidestepping the relaxed setting's noisy witness and the growth it implies for long payment channels; and the hint-free scheme yields larger keys and signatures than optimised Dilithium. Each trades cryptographic optimisation for a transparent, testable artefact, and each maps to an item of section 5.3.

Two limitations attach specifically to the application study. Both ledgers are models rather than nodes, yielding real transaction objects, serialised sizes and the public leak channel on which atomicity depends, but no fees, propagation or

Table 4.2: The six challenges that shaped the implementation. A seventh, cross-cutting difficulty — reproducing the C scheme byte-for-byte in Rust, down to how each language's uniform sampler discards rejected blocks — is discussed with the cross-validation results (section 3.3); the pinned digest is what turned it into a mechanical process. One caveat belongs with the gas figure of challenge 5: the reused ZKNox primitives come from a self-described non-production library, so the measurement is the honest cost of *a* numerically-correct verifier, not an endorsement of that library.

Challenge	The problem	Resolution and lesson
1. The rejection-bound budget	PreSign must reject at $\gamma - \kappa - 1$, one tighter than the basic acceptance bound of eq. (2.1). Set at $\gamma - \kappa$ instead, PreSign, PreVerify and Extract all still succeed, but the *adapted* signature can exceed $\gamma - \kappa$ and be rejected by ordinary Verify — silently, probabilistically, two operations from the cause.	Prevented by reasoning through the norm budget ($\|y\|_\infty \leq 1$, hence $\|\hat{\mathbf{z}} + y\|_\infty \leq \gamma - \kappa$) rather than found by testing.
2. Reference optimisations fight the adaptor identity	The adaptor rests on the exact identity $A\mathbf{z} - c\,t = w + Y$, whereas Power2Round, the hint vector and high/low-bit decomposition deliberately discard low-order information for compactness.	Establishing *which* optimisations to disable was a prerequisite to a working PreVerify, and is why the simplified scheme is a design requirement, not a convenience (section 2.2).
3. Living in another scheme's namespace	The reference globally defines single-letter parameter macros — its `N` is the ring degree, its `K` the module rank — while LAS needs different dimensions with the same natural names, so a constant silently inherits the wrong value through the preprocessor instead of failing to compile.	Forced prefixed parameters and a documented C–Rust–paper name map.
4. Benchmarking a variable-time scheme fairly	Rejection sampling makes sign-class timings heavy-tailed. An acceptance rate inferred from a timing ratio was biased (≈23% where direct counting gives 36.8%), and the two languages' base signing paths initially used different norm-check strategies *inside* the timed loop — identical in function, invisible to the known-answer test, but a work-profile asymmetry biasing the primary comparison.	Attempts are now counted and gated against theory, the drivers mirror each other line-for-line, and every boundary is declared (section 2.6).
5. An error that was self-consistent	Porting the verifier to Solidity, the public matrix A' — stored by the reference *already in the*	Found only by recomputing the challenge digest against the reference's; thereafter

confirmation latency, so on-chain *bytes* are the honest proxy for on-chain cost. And configuration 2 is a deliberate hybrid whose proof system is not post-quantum — included because it is the intermediate point that makes the proof-system comparison possible, not because it is a stack anyone should deploy, since a post-quantum signature does not protect a swap whose fairness rests on a pairing assumption.

Chapter 5

Conclusion, critical reflection, and future work

5.1 Conclusions

This dissertation set out to implement and evaluate a post-quantum exotic signature scheme — a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives — and to demonstrate it in a blockchain atomic swap. That aim was achieved within the stated prototype scope: the lattice core is reused unchanged, with the adaptor layer, wire encoding and application added as new, tested code; the scheme passes functional, serialization, tamper and known-answer tests, is corroborated by an independent Rust implementation reproducing the wire format and the pinned digest byte-for-byte, and drives an end-to-end two-party, two-chain swap.

The central question — what adaptor functionality costs over a plain signature, and what post-quantum operation costs over a classical one — is answered with measured data at declared boundaries. At the core boundary the adaptor layer is almost free in computation, adding at most +10.5% over the basic operation each adaptor operation mirrors, at every parameter set and in both implementations independently. At the packed boundary the premium rises to +20.8–79.5%, not because the arithmetic grows but because each operation must additionally decode a public-key-sized statement Y. The real cost is therefore *communication*, and it is visible on both axes: the signature is 99.3–99.7% response vector, the statement is transmitted on top of it, and that same statement dominates the byte-boundary compute cost. Against the classical ECDSA adaptor the post-quantum price is

likewise paid in bytes rather than adaptor arithmetic — the classical scheme's discrete-logarithm-equality proof makes its own adaptor layer proportionally the more expensive.

Taking the protocol to a UTXO ledger — the venue [6] actually assumes — sharpens that conclusion rather than repeating it, because at the application level the signature is not the dominant cost at all: the role-A proof of knowledge consumes 99.2% of a swap under Groth16 and 98.3% under LaZer. The controlled comparison then inverts the expected trade-off. Changing only the proof system, the *post-quantum* prover is 53.0% faster end-to-end while being 61.9% larger, because Groth16 pays for succinctness in proving work across the whole constraint system whereas the lattice prover, though not succinct, avoids encoding arithmetic modulo a 23-bit prime into a 254-bit pairing-friendly field — and needs no trusted setup. The fully post-quantum configuration is thus at once the faster one, the one with the weaker trust assumption, and the only one Shor does not break, paying for all three in bytes.

All five objectives were met (table 4.1). The principal qualification is scope: the parameter set's concrete security is not formally analysed, and both applications run on modelled ledgers rather than live networks.

5.2 Critical reflection

5.2.1 What was achieved

The central strategic choice — to build *additively* on a standardised reference rather than reimplement lattice arithmetic — paid off twice. It made the implementation tractable in the time available, and it produced an unusually strong provenance argument, the security-critical arithmetic being byte-for-byte the published CRYSTALS-Dilithium reference implementation with a two-branch diff that makes the claim auditable rather than assertable. Being language-agnostic, the strategy also made the second implementation cheap to build, yet it upgraded the correctness argument from one tested codebase to two that agree byte-for-byte and exposed every timing conclusion to an independent harness. If one methodological result deserves to outlive this dissertation, it is that pairing a pinned known-answer digest with an independent reimplementation is a cheap and unusually decisive way to validate a cryptographic port.

Two further things went better than expected. Confronting the fairness problem honestly — comparing LAS against a parameter-matched base rather than optimised Dilithium — strengthened the conclusions rather than weakening them, converting an apparent size deficit into a precisely attributed one. And counting rejection attempts rather than inferring them from a timing ratio corrected a materially wrong earlier figure, a lesson now institutionalised as a gate every benchmark run must pass.

5.2.2 What fell short

Three shortfalls are worth stating plainly, over and above the scoped limitations of section 4.4.

First, and most substantively, **on-chain settlement was measured but not solved**. A complete, numerically-correct native Solidity verifier exists and its cost is known, and a negative result about deployability is still a result — but the honest reading is that demonstrating a *deployable* post-quantum adaptor swap on a smart-contract chain was not achieved: native verification sits far above the per-transaction gas cap, and the optimistic variant only moves the problem into a worst-case fraud proof that also exceeds it. The cause is structural, with two parts. The decisive one is the hash: a full SHAKE256 in EVM bytecode costs roughly 29.4 million gas and on its own exceeds the cap, SHAKE256 not being an EVM-native hash. The lattice arithmetic compounds this rather than independently breaching the cap — the matrix product $A\mathbf{z} - c\,t$ costs roughly 13.9 million gas of execution, below the cap by itself, but carries some 246 kilobytes of calldata that pushes the transaction total higher again. Local optimisation could not close either.

Second, the applications are **exploratory demonstrators, not products**. Both settle over modelled ledgers and assert the properties that matter, but neither runs against a live network. For the UTXO study this is partly forced: Bitcoin Script cannot verify a lattice signature, so the post-quantum configurations could not settle on a real node without a consensus change — exactly the assumption [6] makes in stating its applications over a UTXO chain *whose signature algorithm has been replaced*. The model is faithful to that assumption and yields real transaction objects, sizes and leak channel; what it omits is network, consensus, fee and confirmation overhead. The reported costs are therefore an implementation-level feasibility estimate rather than a deployment figure — and, since the omitted

overheads add cost, certainly not an upper bound on one.

Third, the concrete security of the chosen parameter set is **unanalysed by design**. This was a sanctioned scope decision, not an oversight, but it bounds what the performance numbers mean: they characterise a faithful, testable artefact at a stated engineering setting, not a parameter set anyone should deploy.

5.2.3 What would be done differently

The clearest change is **target selection for the application**. Substantial effort went into taking the verifier on-chain in Solidity before the gas cost was known to be prohibitive. Adaptor signatures are used in practice for swaps on Bitcoin and other UTXO-based chains precisely because the heavy work stays off-chain and the chain meters transactions by *size* rather than execution: Bitcoin still bounds a block, by weight, but has no per-operation gas metering for a verifier to exhaust. The UTXO study bears this out — on that venue the protocol is not merely feasible but unremarkable. The framing was wrong as well as the venue: the project asked "can a post-quantum signature be verified on chain?" when the measurement that mattered was "what dominates a post-quantum swap?", and the answer proved to be a component that never touches a chain at all. Beginning on the UTXO side would have reached that question sooner, with the EVM measurement retained for what it genuinely establishes: the comparative argument for *why* the venue was changed.

Second, **measurement discipline would come first, not second**. Both early artefacts — the biased acceptance estimate and the asymmetric timed loops — followed from building the benchmark before deciding what boundary it measured; declaring the tiers and installing the theory gate at the outset would have cost a day and saved considerably more. Third, the **namespace and provenance discipline** the code now follows was retrofitted only after the preprocessor had silently supplied a wrong constant — on any project modifying a cryptographic reference implementation, that discipline is cheap at the start and expensive to add later.

5.3 Future work

- **Closing the size gap, and why it is not routine.** The response z is almost the whole signature (section 3.4) and already occupies the minimum

fixed width its permitted range admits, so shrinking it requires entropy or range coding, or a parameter redesign with a smaller γ — not merely tighter bit-packing. The reductions that close the gap for plain Dilithium — high/low-bit decomposition with a hint vector, and Power2Round on the public key — are exactly the optimisations this construction had to disable, because they discard low-order information and break the exact identity $A\mathbf{z} - c\,t = w + Y$ that Adapt and Extract depend on (section 2.2). Whether an adaptor-compatible analogue exists is therefore a genuine open question rather than an engineering task, and it is the more interesting half of this item. Either way it would not change the end-to-end swap profile, where the proof remains the bottleneck.

- **Parameter migration.** Move to the $q \approx 2^{24}$ of [6] with a new NTT table and map the parameters to NIST security levels, with before-and-after benchmarks so the trade-off is explicit.

- **Bring on-chain verification within one transaction.** Native verification and an optimistic Naysayer variant [9] both exceed the per-transaction cap in the worst case (section 3.5). Since the hash dispute is what breaches the cap outright and the matrix work is what inflates calldata, the promising routes attack both: a hash dispute avoiding a full SHAKE256 pass or a precompile for it, a Merkle opening of the single disputed matrix row rather than the whole matrix, or a succinct proof of verification — after which a swap should be benchmarked on a live testnet.

- **Take the UTXO study to a live network, as far as one allows.** For the classical configuration *regtest* would validate the integration against a real node's consensus rules but calibrate nothing economic, while *signet* would supply the fees, propagation and confirmation latency the model cannot. For configurations 2 and 3 no Bitcoin test network helps, and that is the point rather than a caveat: signet enforces the same script rules as mainnet, so a deployment figure for the post-quantum swap is blocked on chain support, not on engineering. That asymmetry — a classical swap priceable today against a post-quantum one that cannot be deployed at all — mirrors on the UTXO side the missing-precompile conclusion reached on the EVM.

- **Reduce the proof, not the signature.** The application study identifies the role-A proof as the dominant end-to-end cost by an order of magnitude. Two directions are worth measuring: a *succinct* post-quantum proof system, keeping transparency and quantum resistance while cutting the proof from tens of kilobytes toward Groth16's constant size; and amortisation across swaps, since a party proves knowledge of a fresh statement each time.
- **A new research question.** Extend toward functional adaptor signatures, moving from systems implementation toward a novel construction.

Appendix A

Code listings and reproduction

Per the writing guide, code snippets belong in an appendix rather than the body. Only the most representative listings are included here; the full source is in the project repository.

A.1 Building and reproducing the benchmarks

The evaluation is reproducible from the commands below. The upstream Dilithium reference is pinned at the repository's initial commit (`2374d22`); the classical baseline library is the BlockstreamResearch `secp256k1-zkp` `ecdsa_adaptor` module at commit `95b9835`; the Rust port builds on a vendored copy of the `fips204` crate (commit `c948882`). The C toolchain is the system compiler (`cc`, Ubuntu GCC 13.3.0) with `-O3`; the Rust toolchain is stable `rustc` (the committed run used 1.96.0) in release profile.

Listing A.1: Fair benchmark and tests (first-stage evaluation). The suite script runs everything below, saves the logs and parsed tables as a timestamped evidence run, and regenerates every figure, table, and inline number of this report from that run.

```
# the one supported evidence pipeline (all tests + all benchmarks +
    report sync)
scripts/run_benchmark_suite.sh

# or individually:
cd ref
```

```
# fair, parameter-matched base-vs-adaptor benchmark at each parameter
    set
make test/bench_levels_paper && ./test/bench_levels_paper
make test/bench_levels2 && ./test/bench_levels2
make test/bench_levels3 && ./test/bench_levels3 # target setting
make test/bench_levels5 && ./test/bench_levels5
# correctness, serialization/tamper, known-answer tests
make test/test_las3 && ./test/test_las3
make test/test_serde3 && ./test/test_serde3
make test/test_kat3 && ./test/test_kat3
# end-to-end atomic swap incl. the proof of knowledge pi
# (needs the one-time LaZer build below)
make test/test_zkp3 && ./test/test_zkp3
make test/test_swap3 && ./test/test_swap3
```

Listing A.2: Classical adaptor baseline (one-time clone).

```
git clone --depth 1 https://github.com/BlockstreamResearch/secp256k1-
    zkp \
    third_party/secp256k1-zkp # tested at commit 95b9835
cd ref && make test/bench_classical && ./test/bench_classical
```

Listing A.3: Proof-of-knowledge library (one-time clone and build; see the repository README for the dependency recipe).

```
git clone https://github.com/lazer-crypto/lazer third_party/lazer
cd third_party/lazer && make lazer.h liblazer.a
# the generated proof parameters (ref/relation_zk_params.h) are
    committed,
# so SageMath is needed only to REgenerate them, never to build
```

The Rust port is reproduced with the standard Cargo tooling; the test gate runs first because the benchmarks refuse to time unverified code, and the known-answer test asserts the same pinned digest as the C implementation (`bb6ad0da...260c`).

Listing A.4: Rust port: test gate, statistical benchmark, protocol driver, and size report (one suite script, or the four steps individually).

```
# the one supported Rust evidence pipeline (tests + benchmarks + report
    sync)
scripts/run_rust_bench_suite.sh
```

```
# or individually:
cd rust/fips204-las
cargo test --lib --tests # gate: includes the pinned KAT digest
cargo bench --bench las_bench # Criterion.rs statistical harness
cargo run --release --example bench_levels # protocol driver (mirrors
    the C driver)
cargo run --release --example size_report # packed component sizes
```

A.2 The timing-benchmark procedure

The pseudocode below is the procedure both protocol drivers implement (`ref/test/bench_level` and the Rust `examples/bench_levels.rs`, deliberately line-for-line mirrors), backing the methodology of section 2.6. Ordering matters: correctness is asserted *before* timing on the very objects about to be timed, so no failure path is ever measured, and the rejection gate runs over the *timed* calls themselves, so the gate certifies the same data the tables report. The same procedure runs once per parameter set.

Listing A.5: The timing-benchmark procedure (both tiers). $R = 5$ repetitions; $N = 500$ (sign-class) or 1000 (verify-class) iterations; the rejection gate follows eq. (2.2).

```
Input: parameter set (n, l, kappa); fixed pp seed; fixed 33-byte
   message
 1 Setup: expand A from the fixed seed; KeyGen; relation Gen (Y, y)
 2 Gate 0 (correctness): run the full adaptor contract on the exact
   objects about to be timed (PreVerify accepts; basic Verify rejects
   the pre-signature; Adapt verifies; Extract returns y exactly);
   abort the run on any failure
 3 for each operation op in {KeyGen, Sign, Verify, PreSign, PreVerify,
   Adapt, Extract}: # core tier
 4 for r in 1..R:
 5 a0 <- attempt counter # sign-class only
 6 t0 <- monotonic clock
 7 repeat N times: run op (structures in / structures out;
            fresh masking randomness inside every sign-class call)
 8 run_r <- (clock - t0) / N
```

```
 9 att_r <- (attempt counter - a0) # sign-class only
10 report mean +/- sample SD over run_1..run_R
11 Gate 1 (run validity): for Sign and for PreSign, the mean attempts
    per call over ALL R*N timed calls must satisfy
        |mean - E| <= 5 * E*sqrt(1-1/E) / sqrt(R*N)
    where E = 1/p_acc is the closed-form expectation; abort on failure
12 repeat steps 3-11 at the packed boundary (packed bytes in /
    packed bytes out, validating decode + encode inside the call)
13 emit packed sizes (communication is fixed: measured once, no SD)
```

The Criterion.rs harness measures the same operations under its own protocol (3 s warm-up, 300 samples per operation, outlier classification, bootstrap confidence intervals) and applies the same Gate 1; it shares no timing code with the drivers, which is what makes its agreement an independent check rather than a re-run.

A.3 Methodology details

This section carries the derivations and experimental-design detail deferred from chapter 2, so that the body states the decisions and this appendix justifies them in full.

Why the per-attempt acceptance probability is secret-independent. Each coefficient of the mask is uniform on the $2\gamma + 1$ values of $[-\gamma, \gamma]$, and the secret-dependent term satisfies $\|c\,r\|_\infty \le \kappa$, because the challenge has exactly κ nonzero coefficients of ± 1 and the secret is ternary. Each response coefficient $z_i = y_i + (c\,r)_i$ is therefore uniform on a window of $2\gamma+1$ consecutive integers whose centre is displaced from zero by at most κ. The acceptance interval $[-(\gamma-\kappa),\ \gamma-\kappa]$ is narrower than that window by exactly κ on each side, so it lies wholly inside the window *wherever* the displacement falls. Each coefficient is thus accepted with probability exactly $\big(2(\gamma-\kappa)+1\big)/(2\gamma+1)$, independently of the secret — which is precisely why rejection sampling makes the published response simulatable, and hence why it leaks nothing about r. Raising this to the $(n+\ell)\,d$ coefficients of one attempt gives eq. (2.2).

The run-validity gate in full. Over a run's timed sign-class calls the measured mean attempts $\bar{A}$ must satisfy $|\bar{A} - E| \le 5\,\sigma/\sqrt{n_{\text{calls}}}$, where $E = 1/p_{\text{acc}}$ is the

closed-form expectation of eq. (2.2) and $\sigma = E\sqrt{1 - 1/E}$ is the geometric standard deviation. The test is applied separately for Sign and for PreSign, at both timing tiers, in C and in Rust, and a run that fails it aborts instead of producing evidence. The resulting band is $\pm\sim 8\%$ at the C drivers' 2500 timed calls and $\pm\sim 1\%$ at the Criterion harness's $\approx 10^5$, which is why only the latter resolves the 2% theoretical separation between Sign and PreSign. The gate is a consistency check on the observed rejection rate, not a proof of sampler correctness: it provides evidence that the rejection behaviour is consistent with theory and that rejection-sampling variation is sufficiently controlled for the reported mean.

The Groth16 circuit. No pairing-based proof of this lattice statement was available to reuse, so the circuit is new, and it is far cheaper than the statement suggests. The map $r' \mapsto Ar'$ is linear with public coefficients, so each row is an affine combination of witness variables and needs no witness–witness multiplication — the expensive ingredient in a rank-one constraint system. The public linear map is still enforced, through linear constraints; the only non-linear and range constraints are the ternary bound, enforced as $r'^3 = r'$ per coefficient, and the reduction modulo q, enforced with a range-checked quotient per row. The constraint count is therefore linear in the parameter set's dimensions rather than in their product, and the same statement would have been intractable had it involved products of two *secret* polynomials. Because the matrix is held in the number-theoretic-transform domain and is exported by neither implementation, it is recovered by evaluating the scheme's own linear map on unit vectors, so the circuit encodes the map the signature scheme actually computes with rather than a re-derivation that could drift from it.

Measurement invariants of the UTXO study. Two invariants are enforced rather than assumed. A phase must be timed in *every* run or in none, so that no mean is silently taken over the subset of swaps that reached it; and communication subtotals are computed within each run before being reduced across runs, since summing each message's minimum would describe a swap that never occurred. One-time costs — expanding A, building the elliptic-curve context, and Groth16's trusted setup — are performed before timing begins and reported separately, being amortised over every swap rather than paid per swap. The rejection gate applies here too, but on a dedicated batch of 2000 pre-signature calls rather than

on the two per swap the protocol itself performs: over so few calls the five-sigma band is wider than the distance from the expected attempt rate to a loop that never rejects at all, so a gate on that sample would have been vacuous.

One deliberate exception to reproducibility. Every other input derives from the pinned seed, but Groth16's trusted setup and each of its proofs draw fresh operating-system entropy. Both are security requirements rather than preferences: a reproducible setup would leave the toxic waste recoverable and void soundness, and reusing proof randomness across statements breaks zero-knowledge. The measurement consequence is stated rather than hidden — proof *size* is unaffected, since a Groth16 proof is three group elements regardless, but proving *time* includes the entropy draw and so carries a little extra run-to-run variance.

A.4 The modelled UTXO ledger

The ledger behind section 2.7 is a model, not a node: it has no peer-to-peer layer, no mempool policy, no fee market, no reorganisations and no script interpreter. What it does provide is what the study needs — real transaction objects with canonical serialisation, a signature hash computed over the transaction rather than over an abstract message, and the public observability on which the protocol's atomicity depends, since a settled transaction's signature can be read back off the ledger by anyone, which is exactly how the witness leaks. An output carries an amount plus a spending condition: a signature check under a public key, with an optional timelocked refund branch.

Deploying instead on a real Bitcoin node is not a matter of effort. Bitcoin Script cannot verify a lattice signature, so configurations 2 and 3 could not settle there without a consensus change — which is precisely the assumption [6] makes when it states its applications over a UTXO-based chain *whose signature algorithm is replaced with a lattice-based signature scheme*, and the reason that sentence is stated as an assumption rather than left implicit.

A.5 Wire encoding and the validating decoder

The packed field widths behind section 2.3 are 23 bits per coefficient for the public key and statement, 2 bits for the ternary secret and witness, and 18 or 19 bits

for the response z depending on the parameter set, since that field must hold $2(\gamma - \kappa)$ and γ grows with $\kappa\, d\, (n + \ell)$. Because a verifier cannot trust its input, the decoder is defensive: it rejects a public-key coefficient $\geq q$, the invalid ternary code, and any response field outside the parameter set's band, while the encoder symmetrically rejects a non-ternary secret or a response exceeding $\gamma - \kappa$. This is the interface a Solidity, precompile or zero-knowledge-circuit verifier would expose, and it is the object the tamper test of section 3.1 attacks.

One encoding decision is worth recording. Following FIPS 204 [13], the wire format carries not the challenge polynomial c but only the 32-byte commitment hash $\tilde{c}$ from which it is derived, every consumer re-deriving $c = \mathsf{SampleInBall}(\tilde{c})$ after decoding. This is a computation-versus-storage trade: storing the expanded polynomial would save the re-derivation at every decode, while storing the seed keeps the signature smaller and matches the standard's format. Where the same signature is decoded repeatedly the expanded challenge can be cached after the first derivation, making the re-derivation a one-time cost per object rather than a per-use one.

A.6 The atomic-swap demonstration and its proof of knowledge

The narrated demonstration referenced in section 3.5 follows Fig. 1 of [6] *verbatim*. Two parties swap coins across two ledgers with no trusted third party and no on-chain scripts. Alice, the witness holder, commits first: she draws a fresh statement/witness pair (Y, y), produces a zero-knowledge proof π that Y has a *ternary* preimage $(\exists\, r' : \mathbf{A}r' = Y \wedge \|r'\|_\infty \leq 1)$, pre-signs the transaction spending her own coin, and sends $\{Y, \pi, \hat{\sigma}_1\}$ in one off-chain message. Bob pre-signs his own transaction against the *same* Y only after both π and Alice's pre-signature verify — the protocol's abort gate; at that point neither pre-signature is spendable. Alice, who knows y, adapts and publishes her signature to claim Bob's coin; the act of publishing reveals y, which Bob extracts and uses to adapt and publish his own signature. Either both legs settle or neither does. The demonstration prints this narrative and asserts every step, including the counterfactuals that a pre-adaptation signature fails basic verification and that π is rejected against any other statement.

The proof π reuses the LaZer lattice zero-knowledge library [10] rather than a

proof system written from scratch — the same reuse posture applied throughout. Because LaZer proves per-partition binary coefficients or exact Euclidean norms, the ternary statement is encoded by binary decomposition, $[\mathbf{A} \mid -\mathbf{A}]\,(r_+ \,\|\, r_-) = Y$ with both halves proven binary, which is equivalent to knowledge of a ternary preimage $r' = r_+ - r_-$. The generated parameter set reports a knowledge error below 2^{-127} under Module-SIS and a proof size of approximately 31 kilobytes, with proving and verifying each completing well under a second. A dedicated test suite asserts completeness, rejection of every sampled single-byte tamper, rejection against a wrong statement, and that the prover refuses a non-ternary witness. A small ledger abstraction — accounts with balances, a block height, and adaptor-locked contracts, the scriptless analogue of a hash-time-locked contract with the hash lock replaced by the statement Y and the time lock by a timeout height — additionally carries three asserted scenarios: the cross-chain happy path, a timeout/refund in which a premature refund is rejected and both parties recover their funds, and a multi-hop payment.

A.7 The optimistic (Naysayer) on-chain verifier

Because the native Solidity verifier exceeds the per-transaction gas cap (section 3.5), an *optimistic* alternative was also implemented and measured: `LASNaysayer`, a poqeth-inspired Naysayer verifier [9] for the lattice setting — poqeth itself implements only hash-based and multivariate schemes and rules Dilithium out on public-key size. Instead of re-executing verification, the contract accepts a settlement optimistically and lets any party dispute a single failing step within a challenge window. Verification is decomposed into three locally checkable invariants — the response norm, each recomputed commitment coefficient (with the challenge derived on chain), and the final hash — so a dispute recomputes only one coefficient by schoolbook convolution and never runs the number-theoretic-transform batch. An adversarial suite of nine tests passes, covering soundness (no dispute voids a valid settlement), completeness for each fault type (including a C-exported pure hash-fault vector against which only the hash dispute succeeds), the challenge-window and replay guards, input validation, and pull-payment isolation against a hostile recipient.

The result is deliberately reported as mixed. Honest settlement is cheap — an optimistic claim costs approximately 1.1 million gas and finalisation approximately

27 000, since the batch never executes on the happy path — but the binding cost is the worst-case fraud proof. The norm dispute costs approximately 34 000 gas; the commitment dispute, in a baseline that passes the whole public matrix in calldata, approximately 13.9 million gas of execution together with an approximately 246-kilobyte calldata payload; and the hash dispute approximately 29.4 million gas. These are execution-gas figures, so the full per-transaction totals — adding the fixed intrinsic cost and the calldata — are strictly larger. The hash dispute alone therefore exceeds the EIP-7825 cap of 16 777 216 by roughly 1.75× on execution gas before any calldata, because it must run a complete SHAKE256 in Solidity, which is not an EVM-native hash — unlike the Keccak-based schemes of [9]. The optimistic mode thus moves cost off the honest path but does *not*, in this baseline, bring the worst-case dispute within a single transaction; doing so would require opening only the single disputed matrix row through a Merkle commitment rather than sending the whole matrix, and a hash dispute that avoids a full SHAKE256 pass.

A.8 Full benchmark data tables

The tables in chapter 3 carry the primary results; this section gives the supporting data at every parameter set, produced by the commands above (listing A.1). Table A.1 lists every core-tier per-operation timing with its standard deviation (the data plotted in fig. 3.1, and the source of the core block of table 3.2); table A.2 gives the component byte sizes at every parameter set (the target setting is the body table table 3.5).

Figure A.1 is the parameter-sensitivity check referred to in section 3.2: it repeats the primary base-versus-adaptor comparison at all four parameter sets and confirms that the core-tier adaptor premium stays in the single digits as the dimensions are scaled up, so the headline result is not specific to the target setting.

A.9 Auditing the reuse claim

The “reused vs. modified vs. added” claim (table 2.1) is verified by diffing the pristine baseline branch against the project branch.

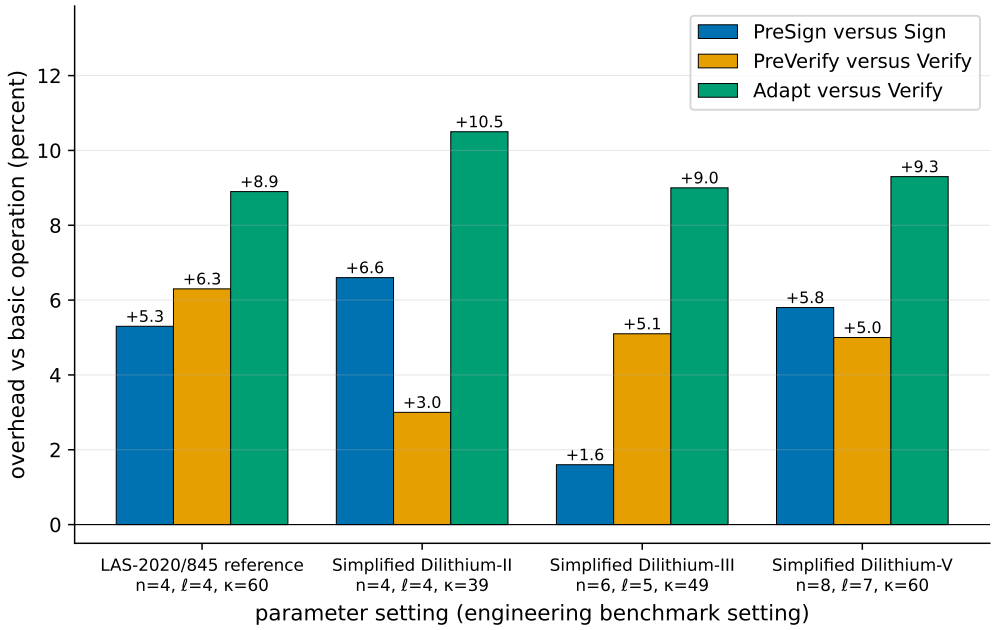

Figure A.1: Parameter sensitivity of the adaptor overhead: each LAS adaptor operation as a percentage over the basic operation it mirrors, at the four parameter sets of table 2.4. PreSign is paired with Sign; PreVerify and Adapt with Verify; Extract has no basic analogue and is omitted. The core-tier premium stays in the single digits everywhere, so adding adaptor functionality does not become more expensive as the scheme is scaled up. This is a secondary robustness check; the primary comparison is the per-operation overhead at the target setting, table 3.2.

Listing A.6: Branch diff proving the lattice core is untouched (no output means identical).

```
git diff --name-status dilithium-baseline main -- ref/
git diff dilithium-baseline main -- ref/poly.c ref/ntt.c ref/sign.c \
    ref/packing.c ref/polyvec.c ref/reduce.c ref/rounding.c ref/params.h
```

A.10 Selected code listing: Adapt and Extract

The adaptor mechanism reduces to two short routines. ADAPT adds the witness to the pre-signature's response; EXTRACT recovers the witness by subtraction and confirms it against the statement. Both reuse the reference's polynomial arithmetic verbatim.

Listing A.7: `las_adapt` and `las_ext` from `ref/las.c`.

Table A.1: Per-operation timing, reported independently (μs per operation; mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500 sign-class / 1000 verify-class iterations; `bench_levels`, machine of record in section 2.6). KeyGen, Sign, and Verify *are* the basic signature and are reused unchanged by LAS; PreSign, PreVerify, Adapt, and Extract are the four adaptor operations. Public-parameter Setup, a one-time global cost, runs in 46–169 μs and is omitted. Parameter sets and symbols: tables 2.4 and 2.5.

Operation	LAS-2020/845 reference $(4, 4, 60)$	Simplified Dilithium-II $(4, 4, 39)$	Simplified Dilithium-III $(6, 5, 49)$	Simplified Dilithium-V $(8, 7, 60)$
Basic signature (reused unchanged by LAS)				
KeyGen	59.0 ± 1.7	61.7 ± 0.8	91.6 ± 2.0	133 ± 7
Sign	471 ± 24	570 ± 20	852 ± 25	991 ± 43
Verify	126 ± 3	131 ± 3	199 ± 6	265 ± 12
LAS adaptor operations				
PreSign	496 ± 21	608 ± 35	866 ± 41	1049 ± 51
PreVerify	134 ± 6	135 ± 2	210 ± 9	278 ± 10
Adapt	137 ± 7	144 ± 7	217 ± 5	290 ± 8
Extract	47.1 ± 0.7	55.6 ± 4.2	84.2 ± 6.3	117 ± 6

```
int las_adapt(las_sig *sig, const las_sig *presig, const uint8_t *m,
    size_t mlen,
              const las_pk *Y, const las_sk *y, const las_pk *pk, const
                  las_pp *pp) {
  unsigned int j;
  if(las_preverify(presig, m, mlen, Y, pk, pp))
    return -1;
  sig->c = presig->c;
  for(j = 0; j < LAS_M; ++j) { /* z = z_hat + y_witness */
    poly_add(&sig->z[j], &presig->z[j], &y->s[j]);
    poly_reduce(&sig->z[j]);
  }
  return 0;
}

int las_ext(las_sk *y, const las_sig *sig, const las_sig *presig,
            const las_pk *Y, const las_pp *pp) {
  poly Ay[LAS_N];
  unsigned int j;
```

Table A.2: LAS objects by component (packed bytes; `bench_levels`). The LAS-2020/845 reference set shares the Simplified Dilithium-II dimensions. The challenge c is fixed; the response z grows with $n + \ell$ and dominates the signature at every setting. The target-setting column is the body table table 3.5.

Component	Simplified Dilithium-II $(4, 4)$	Simplified Dilithium-III $(6, 5)$	Simplified Dilithium-V $(8, 7)$
c (challenge)	32	32	32
z (response)	4608	6688	9120
Signature (c, z)	4640	6720	9152
z as % of signature	99.3%	99.5%	99.7%
Public key pk	2944	4416	5888
Secret key sk	512	704	960
Statement Y (LAS only)	2944	4416	5888
Witness y (LAS only)	512	704	960
Pre-signature (LAS only)	4640	6720	9152

```
  for(j = 0; j < LAS_M; ++j) { /* s = z - z_hat */
    poly_sub(&y->s[j], &sig->z[j], &presig->z[j]);
    poly_reduce(&y->s[j]);
  }
  las_Amul(Ay, pp, y->s); /* check A*s == Y */
  for(j = 0; j < LAS_N; ++j)
    if(!poly_equal(&Ay[j], &Y->t[j]))
      return -1;
  return 0;
}
```

Bibliography

[1] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 635–664. Springer, 2021.

[2] Blockstream Research. secp256k1-zkp: Ecdsa adaptor signature module. Software library, 2024. `https://github.com/BlockstreamResearch/secp256k1-zkp`, commit 95b9835.

[3] Maxime Buser, Rafael Dowsley, Muhammed Esgin, Clémentine Gritti, Shabnam Kasra Kermanshahi, Veronika Kuchta, Jason Legrow, Joseph Liu, Raphaël Phan, Amin Sakzad, et al. A survey on exotic signatures for post-quantum blockchain: Challenges and research directions. *ACM Computing Surveys*, 55(12):1–32, 2023.

[4] CRYSTALS team. CRYSTALS-Dilithium reference implementation. Software repository, 2023. `https://github.com/pq-crystals/dilithium`, commit 2374d22.

[5] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR transactions on cryptographic hardware and embedded systems*, pages 238–268, 2018.

[6] Muhammed F Esgin, Oğuzhan Ersoy, and Zekeriya Erkin. Post-quantum adaptor signatures and payment channel networks. In *European Symposium on Research in Computer Security*, pages 378–397. Springer, 2020.

[7] Lloyd Fournier. One-time verifiably encrypted signatures a.k.a. adaptor signatures. Manuscript, 2019. `https://github.com/LLFourn/one-time-VES`.

[8] Brook Heisler, Jorge Aparicio, and contributors. Criterion.rs: statistics-driven micro-benchmarking for Rust. Software repository, 2025. `https://github.com/criterion-rs/criterion.rs`, version 0.8.2.

[9] Ruslan Kysil, István András Seres, Péter Kutas, and Nándor Kelecsényi. poqeth: Efficient, post-quantum signature verification on ethereum. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 327–343, 2025.

[10] Lazer-crypto project. LaZer: a library for lattice-based zero-knowledge proofs. Software library, 2025. `https://github.com/lazer-crypto/lazer`; accompanies the CCS 2024 paper "The LaZer Library: Lattice-Based Zero Knowledge and Succinct Proofs for Quantum-Safe Privacy".

[11] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–755. Springer, 2012.

[12] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. Anonymous multi-hop locks for blockchain scalability and interoperability. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019*, 2019.

[13] National Institute of Standards, Technology (NIST), Thinh Dang, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, and Angela Robinson. Module-lattice-based digital signature standard, August 2024.

[14] Andrew Poelstra. Scriptless scripts. Presentation, Milan Bitcoin meetup / Blockstream, 2017. `https://github.com/apoelstra/scriptless-scripts`.

[15] Sebastien Riou, Jong-Yeon Park, Liga Anwar, Axel Poschmann, and Michael Hutter. Apples, oranges, and signatures: Pitfalls and methodology in ML-DSA benchmarking. Cryptology ePrint Archive, Paper 2026/1333, 2026. `https://eprint.iacr.org/2026/1333`.

[16] Eric Schorn. fips204: FIPS 204 ML-DSA in pure Rust. Software repository, 2025. `https://github.com/integritychain/fips204`, vendored at commit c948882.

[17] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.